A project report on

VESSEL GUIDED DEEP LEARNING FOR RARE RETINAL DISEASE DETECTION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in COMPUTER SCIENCE ENGINEERING WITH SPECIALIZATION IN DATA SCIENCE

by

K Dhiraj 23BDS1096

Apurva Sahay 23BDS1115

M Vaishnavi 23BDS1091



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled "**VESSEL GUIDED DEEP LEARNING FOR RARE RETINAL DISEASE DETECTION**" submitted by K Dhiraj (23BDS1096), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Modigari Narendra.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: Signature of the Candidate

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"VESSEL GUIDED DEEP LEARNING FOR RARE RETINAL DISEASE DETECTION"** is prepared and submitted by **K Dhiraj (23BDS1096)** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science with Specialisation in Data Science** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:
Name: Dr./Prof.
Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,
Computer Science and Engineering
With specialisation in Data Science
Name: **Dr. Vergin Raja Sarobin**
Date:

ABSTRACT

Retinal diseases are a major cause of preventable vision loss, and automated fundus screening needs classifiers that cope with many diseases at once, with rare diseases, and with unreliable confidence scores. This report studies vessel-guided curriculum learning with uncertainty calibration for long-tailed multi-label fundus classification on the RFMiD dataset (3,200 images). Twenty-five labels with at least ten training positives were retained, with an imbalance of up to 138:1 between the most and least frequent classes.

The proposed system combines four components. The first is a vessel prior, a frozen HRM-UNet vessel segmenter. The second is a DenseNet-121 classifier, which receives the vessel map through learnable, zero-initialised gates at three intermediate scales. The third is a two-level soft curriculum that weights each sample's loss by class difficulty and sample difficulty. The fourth is a calibration stage: post-hoc temperature scaling followed by class-wise scaling, with Monte Carlo dropout providing uncertainty-based triage.

Evaluation used five-fold multilabel-stratified cross-validation on the pooled training and validation images, paired bootstrap comparisons, and a held-out 640-image official test set.

The results are mixed. The best recipe was a plain DenseNet-121 without vessel guidance, trained with the older 30-epoch, lightly regularised settings, reaching an out-of-fold mAP of 0.634 and a test-ensemble mAP of 0.622 (AUC 0.954). Vessel guidance gave no statistically detectable change (ΔmAP −0.007, 95% CI [−0.020, +0.006]), and the curriculum reduced mAP (−0.031). The learned gate parameters were negative, and the curriculum’s vessel-agreement signal was nearly constant, so the vessel prior was not used as intended. Calibration was effective: it cut expected calibration error from 0.035–0.075 to below 0.006 for every recipe. Four of the 25 classes were never detected at the operating thresholds, and tail-class results rest on only 3–7 test positives each. The work contributes a reproducible long-tailed evaluation protocol, a calibrated multi-label baseline, and documented negative findings on vessel guidance and curriculum learning.

Keywords: retinal fundus classification, long-tailed learning, vessel guidance, curriculum learning, uncertainty calibration, RFMiD

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Modigari Narendra, Assistant Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Computer Vision and AI.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Dr. Vergin Raja Sarobin M**, Head of the Department, B.Tech.Computer Science Engineering with specialisation in Data Science and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **K Dhiraj**

CONTENTS

Chapter No.	Title	Page No.
	Abstract	1
	Acknowledgement	2
	Contents	3-4
	List of Figures	5
	List of Tables	6-7
	List of Program Listings	8
1	INTRODUCTION	1
1.1	Introduction	1
1.2	Motivation	2
1.3	Problem Statement	3
1.4	Objectives	3
1.5	Scope of the Project	4
2	LITERATURE SURVEY	5
2.1	Introduction	5
2.2	Survey of Existing Work	5
2.3	Research Gaps	14
3	PROPOSED SYSTEM AND METHODOLOGY	17
3.1	Overview	17
3.2	System Architecture	18
3.3	Methodology	27
4	IMPLEMENTATION	36
4.1	Tools and Technologies	36
4.2	Implementation Details	37
5	RESULTS AND DISCUSSION	44

5.1	Experimental Setup	44
5.2	Results	46
5.3	Discussion	57
6	CONCLUSION AND FUTURE WORK	60
6.1	Conclusion	60
6.2	Future Work	63
	REFERENCES	64
	APPENDICES	67
Appendix A	Experimental Environment and Reproducibility	67
Appendix B	Hyperparameter Configuration	67
Appendix C	Dataset Details and Label Abbreviations	69
Appendix D	Code Listings	69
Appendix E	Complete Results for All Recipes	80
Appendix F	Per-Class Out-of-Fold Average Precision	81
Appendix G	Per-Class Test Results	82
Appendix H	Key Formulations	83

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1.1	Colour fundus image showing major retinal structures and example lesions	2
Figure 3.1	Overall architecture of the proposed vessel-guided classification system	19
Figure 3.2	Preprocessing pipeline and the two aligned views	20
Figure 3.3	HRM-UNet vessel segmentation architecture	21
Figure 3.4	Multi-scale vessel-gated DenseNet-121	23
Figure 3.5	Two-level curriculum scheduler	25
Figure 3.6	Calibration and uncertainty pipeline	26
Figure 3.7	Example vessel maps produced by the frozen HRM-UNet	29
Figure 3.8	Grad-CAM overlays and vessel-agreement analysis	34
Figure 4.1	Example vessel maps from the frozen HRM-UNet	39
Figure 5.1	Validation mAP and training-loss curves for the Stage 1 recipes	46
Figure 5.2	Overview comparison of recipes	47
Figure 5.3	Grad-CAM overlays for diabetic retinopathy (vascular class)	54
Figure 5.4	Grad-CAM overlays for macular hole (non-vascular class)	55
Figure 5.5	Uncertainty-based deferral curve	55

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Recent fundus classification studies reviewed	6
Table 2.2	Backbone evidence in the reviewed studies	7
Table 2.3	Use of vessel information in the reviewed studies	9
Table 2.4	Coverage of the key components across the reviewed studies	13
Table 2.5	Research gaps and the corresponding response in this work	16
Table 3.1	Stages of the proposed system	18
Table 3.2	Difficulty signals of the two-level curriculum	24
Table 3.3	Dataset summary after preprocessing of labels	27
Table 3.4	Cached arrays	28
Table 3.5	Training configuration	30
Table 3.6	Experimental stages and research questions	32
Table 3.7	Software and hardware environment	35
Table 4.1	Computing environment	36
Table 4.2	Datasets and pretrained components	36
Table 5.1	Training cost per fold on a single T4 GPU	44
Table 5.2	Recipes and the question each addresses	45
Table 5.3	Stage 1 results, out-of-fold (n = 2,560) and test-split ensemble (n = 640)	46
Table 5.4	Paired bootstrap on OOF mAP, each recipe against A_baseline	47
Table 5.5	Input-resolution sweep for B_vessel (five-fold)	48

Table 5.6	RETFound trial on fold 0 (512 held-out images)	49
Table 5.7	One-at-a-time ablation on a vessel + ASL + mixup + SWA base (folds 0–2)	50
Table 5.8	Expected calibration error (equal-mass bins) before and after post-hoc calibration	51
Table 5.9	Test-split metrics, five-model ensemble	51
Table 5.10	Per-class test results, B_vessel_384px (threshold chosen on OOF predictions)	52
Table 5.11	Comparison with published work	56
Table 5.12	Outcome of each hypothesis	57
Table 6.1	Paired bootstrap comparisons on out-of-fold mAP	60
Table A.1	Environment and reproducibility settings	67
Table B.1	Main hyperparameters (v8 regime)	67
Table B.2	Recipe definitions	68
Table E.1	Summary of all recipes	80
Table F.1	Per-class OOF average precision	81
Table G.1	Per-class test metrics (B_vessel_384px)	82

LIST OF PROGRAM LISTINGS

Listing No.	Listing Title	Page No.
Code Snippet D.1	VesselGate module and VGDenseNet forward pass	69
Code Snippet D.2	Tiled sliding-window vessel segmentation	70
Code Snippet D.3	Recipe dataclass defining a training run	74
Code Snippet D.4	preprocess_pair with thread-local CLAHE	74
Code Snippet D.5	Tiled sliding-window vessel segmentation	75
Code Snippet D.6	Joint geometric augmentation of image and vessel map	75
Code Snippet D.7	VGDenseNet.param_groups (layer-wise learning-rate decay)	76
Code Snippet D.8	train_one_epoch with mixed precision, mixup and accumulation	77
Code Snippet D.9	TemperatureScaler, VectorScaler and fit_chain	78
Code Snippet D.10	GradCAM class and cam_vessel_correlation	80

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The retina is the light-sensitive tissue at the back of the eye, and many retinal diseases progress without early symptoms. Conditions such as diabetic retinopathy (DR), age-related macular degeneration (ARMD), glaucoma-related changes, retinal vein occlusion and macular hole can cause irreversible vision loss if left untreated. Colour fundus photography is a non-invasive, inexpensive and widely available imaging technique, making it suitable for large-scale retinal screening.

Fundus image interpretation requires trained ophthalmologists, who remain scarce in rural and underserved regions. This has motivated the development of computer-aided diagnosis systems. Deep convolutional neural networks (CNNs) have achieved specialist-level performance on specific tasks, but most early systems focused on a single disease. Since one image may contain multiple conditions, research has shifted towards multi-label classification.

The Retinal Fundus Multi-disease Image Dataset (RFMiD) supports this task with 3,200 fundus images annotated with normal/abnormal labels and 45 disease labels, divided into official training, validation and test splits.

Two developments are particularly relevant to this project: anatomical priors and reliable uncertainty estimation. Retinal vessels provide useful structural information, and segmentation networks trained on datasets such as DRIVE can extract vessel maps. Meanwhile, probability calibration and uncertainty estimation help determine whether a model's predictions are trustworthy.

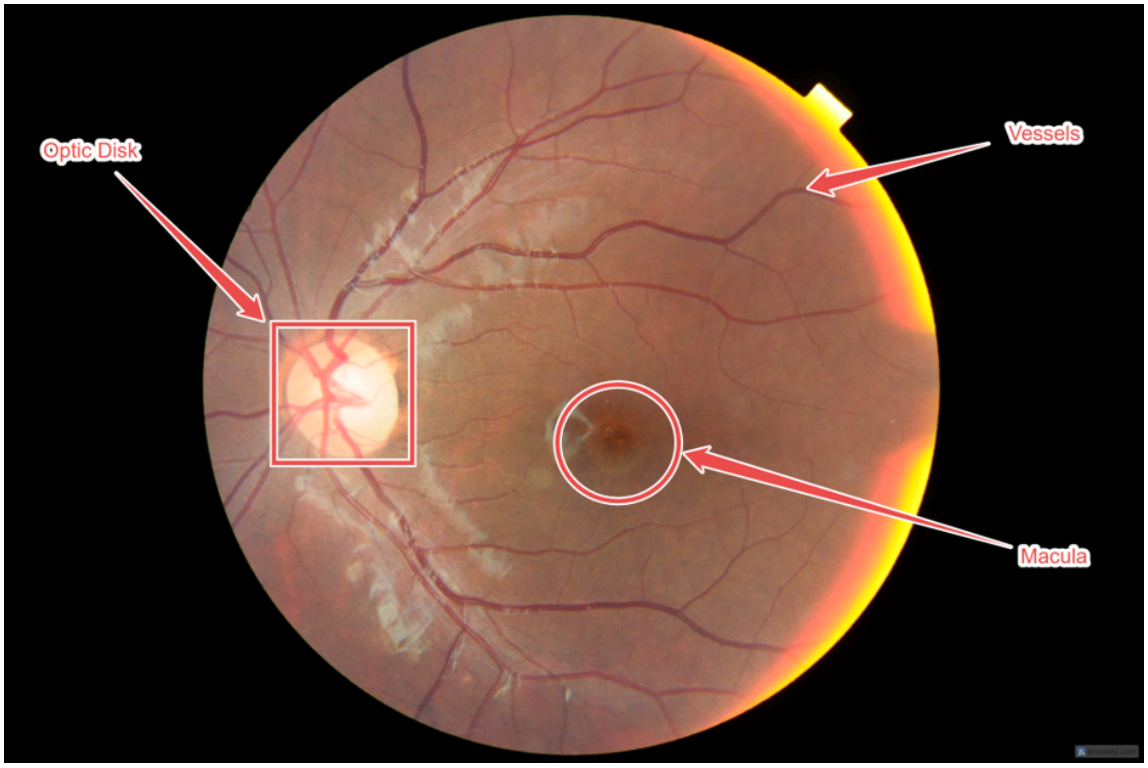


Figure 1.1: Colour fundus image showing major retinal structures and example lesions

1.2 MOTIVATION

The project is motivated by the following observations:

- **Clinical need:** Automated multi-disease screening can extend retinal assessment to regions with limited access to specialists.
- **Class imbalance:** In RFMiD, the most frequently retained class outnumbers the rarest by 138:1. Models may perform well on common diseases while missing rare but important conditions.
- **Anatomical information:** Vessel abnormalities indicate vascular diseases such as DR and retinal vein occlusion. Explicit vessel guidance may help classification, but must remain flexible because many diseases are non-vascular.
- **Unreliable confidence:** Deep networks can be over-confident. Calibrated probabilities and uncertainty estimates can help identify cases requiring human

review.

- **Reliable evaluation:** Rare classes have very few test positives, making small performance differences potentially unreliable. Statistical evaluation is therefore essential.

1.3 PROBLEM STATEMENT

Given a colour fundus photograph, the objective is to predict the presence of multiple retinal conditions. The task is challenging because of four factors:

- **Severe class imbalance:** Long-tailed labels bias models towards common diseases.
- **Limited data:** Approximately 2,000 training images and few positive examples for rare classes increase overfitting.
- **Heterogeneous disease evidence:** The model must exploit vessel abnormalities where relevant without depending on them for every disease.
- **Unreliable uncertainty:** Raw predictions may be poorly calibrated, while conventional metrics can conceal miscalibration in rare classes.

This project aims to develop and rigorously evaluate a multi-disease classifier that addresses class imbalance, incorporates flexible vessel guidance and produces calibrated, uncertainty-aware predictions, with particular attention to reliable rare-class evaluation.

1.4 OBJECTIVES

The main objective is to develop a vessel-guided, long-tail-aware and uncertainty-calibrated multi-label retinal classifier and evaluate it against a strong baseline.

The specific objectives are:

- Develop preprocessing pipelines that maintain spatial alignment between fundus

images and vessel maps.

- Generate vessel priors using a frozen HRM-UNet pretrained on DRIVE and fuse them into DenseNet-121 through multi-scale, residual, learnable attention gates.
- Address class imbalance using capped inverse-frequency weighting and deferred re-weighting, while investigating Asymmetric Loss, mixup, stochastic weight averaging and a two-level curriculum scheduler.
- Improve probability reliability using post-hoc temperature scaling and class-wise scaling, and estimate uncertainty through Monte-Carlo dropout.
- Establish a statistically sound evaluation protocol using multilabel-stratified five-fold cross-validation, out-of-fold calibration, paired bootstrap comparisons and a held-out official test split.
- Evaluate individual components through controlled ablations, including input resolution and a fundus-pretrained transformer backbone, using ranking, long-tail, threshold-based and calibration metrics.
- Assess interpretability by comparing Grad-CAM activation maps with vessel priors to examine the effect of vessel guidance.

1.5 SCOPE OF THE PROJECT

Within Scope

- Multi-label classification using RFMiD, restricted to 25 classes with at least 10 positives in the original training split.
- A frozen HRM-UNet pretrained on DRIVE for vessel guidance, DenseNet-121 classification and an experimental RETFound ViT-L/16 backbone.
- Long-tail handling, curriculum learning, post-hoc calibration, Monte-Carlo dropout uncertainty and Grad-CAM interpretability.
- Cross-validation on pooled training and validation data, a held-out test split and component-wise ablation studies.

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

This chapter reviews four key research areas underlying the project: deep-learning-based retinal fundus classification, long-tailed class imbalance, vessel-guided learning and predictive confidence calibration. Although each area has been studied extensively, their integration remains limited.

The review covers recent (2024–2026) studies on fundus classification using RFMiD and APTOS 2019, lightweight retinal models, imbalance handling, vessel-guided methods, and diabetic retinopathy and OCT analysis. It also examines vessel segmentation techniques used to generate anatomical priors. Reported results are retained as published; however, differences in datasets, splits, class definitions and metrics limit direct comparisons.

Section 2.2 reviews existing methods by theme, while Section 2.3 identifies the research gaps and explains how the present project addresses them.

2.2 SURVEY OF EXISTING WORK

2.2.1 Deep learning for retinal fundus disease classification

Transfer-learned convolutional networks are the dominant approach to fundus classification. A recent survey of lightweight models (Al-johani et al., 2026) reports that classic CNNs such as VGG, ResNet, DenseNet and Inception typically reach 90 to 96% accuracy on benchmark data, hybrid CNNs 92 to 97% on multi-label tasks, transformers 95 to 99% (mainly on OCT), and lightweight networks such as MobileNet and EfficientNet-B0 85 to 94%. The same survey identifies RFMiD as the hardest benchmark among those it covers, because of its 46 labels, label overlap and severe imbalance, and notes that cross-dataset evaluation typically loses 10 to 20% accuracy through domain shift.

Four experimental studies on RFMiD or APTOS 2019 are most relevant and are

summarised in Table 2.1.

Table 2.1: Recent fundus classification studies reviewed

Study	Task and data	Method	Main reported result	Limitations noted
Perumalsamy et al. (2026), CLARE-Net	RFMiD converted to 8-class single-label	EfficientNet-B3 and V2-S; F1-weighted adaptive ensemble; per-class thresholds; ECE reported	Acc. 75.38%, macro-F1 0.7428, ECE 0.0649	Two ensemble members; DN sensitivity 0.37; n uncalibrated ECE baseline; calibrati method unspecifie
Sekhar et al. (2026)	APTOS 2019; binary DR detection and 5-grade severity	DenseNet-121, focal loss, Grad-CAM	Binary acc. 89.09%; severity acc. 82.26%, macro-F1 about 0.60	Weak severe/PDR performance; som reported figures appear mutually inconsistent; singl dataset
Venkatappa et al. (2026)	RFMiD 1.0 + 2.0, 10-class single-label, 1,098 images	ResNet-34, DenseNet-121, VGG-19, Inception-v4 under identical settings; targeted augmentation; four XAI methods	DenseNet-121: acc. 88.48%, macro-F1 0.697	165 test images, 2 3 per rare class; n calibration; XAI methods applied t different models
Anusha et al. (2026)	RFMiD, 46-label multi-label (37 classes evaluated)	Two-tier EfficientNetB0 (healthy vs diseased, then multi-label with CBAM and weighted BCE); Grad-CAM++	Tier-2 per-class accuracy 0.97	Metric inflated by true negatives; no macro-F1 or per-class AUC; 8 classes without te samples; no ablati
Al-johani et al. (2026), survey	ODIR, RFMiD, APTOS, EyePACS and others, 2020 to 2025	PRISMA-style review of lightweight models	CLAHE about +3 to 7%; U-Net segmentation about +2 to 5%	Figures not directl comparable acros studies; calls for macro-averaged, class-wise metrics

CLARE-Net combines two EfficientNet models, weighting each by its validation macro-F1 and tuning a decision threshold for every class. It improves on either single model and is the only study in this group that reports expected calibration error (ECE), but its per-class results show that the tail remains weak, for example a sensitivity of 0.37 for the DN class. Sekhar et al. split DR grading into a binary detector followed by a five-level grader; the binary detector has very high recall (0.9889), which suits screening, but macro-F1 for the five grades is far below accuracy, indicating poor performance on the rare severe grades. Venkatappa et al. provide the most controlled comparison of backbones and show large gains from targeted augmentation of rare classes, but with only 165 test images and two or three test samples in several classes, individual per-class changes (for instance macular hole falling from 0.50 to 0.00 F1 on two samples) are largely noise. Anusha et al. are closest to the multi-label, long-tailed setting of the present work, but report per-class accuracy averaged over a sigmoid output, which is dominated by true negatives in a sparse multi-label problem and is therefore not comparable with multi-class accuracy.

2.2.2 Backbone selection

The choice of backbone determines both accuracy and the ease with which additional signals can be injected. The only controlled multi-backbone comparison among the reviewed studies (Venkatappa et al., 2026) favours DenseNet-121, with the highest accuracy and macro-F1 of the four networks tested. Table 2.2 collects the available evidence.

Table 2.2: Backbone evidence in the reviewed studies

Backbone	Where tested	Headline result	Takeaway
DenseNet-121	Venkatappa et al. (RFMiD, 10-class); Sekhar et al. (APTOS)	88.48% acc., macro-F1 0.697 (best head-to-head)	Strongest single backbone in the only controlled comparison; about 8M parameters
EfficientNet-B3 / V2-S	CLARE-Net	71.5 to 72.1% acc., macro-F1 0.70 to	Solid alone, better as an Sadaptive ensemble

		0.71	
EfficientNet-B0 (+CBAM)	Anusha et al.	High per-class accuracy (multi-label)	Efficient and attention-friendly; headline metric inflated
ResNet-34	Venkatappa et al.	85.45%, macro-F1 0.648	Good, below DenseNet-121
Inception-v4	Venkatappa et al.	83.03%, macro-F1 0.596	Best before augmentation; small gains after
VGG-19	Venkatappa et al.	76.97%, macro-F1 0.599	Weakest and most overfitting-prone (143.7M parameters)
MobileNetV3, ShuffleNet	Survey only	85 to 94% (cross-study)	Best deployability; not tested in any experimental study

Dense connectivity is also attractive for this project because every layer receives the feature maps of all preceding layers, giving a vessel-guidance signal several points at which to enter the feature hierarchy. Larger models are an alternative: the survey notes that foundation models such as RETFound generalise best but are too large for most deployments.

2.2.3 Handling Class Imbalance and Long-Tailed Labels

The reviewed studies report poor minority-class performance and address imbalance using static methods, including validation-based ensemble weights and class thresholds (Perumalsamy et al.), focal loss (Sekhar et al.; Anusha et al.), weighted binary cross-entropy (Anusha et al.), and targeted augmentation that expands rare classes by 200% (Venkatappa et al.). Mane and Shekapure (2025) use GAN-based synthesis and federated averaging, while Kadhim et al. (2024) address a 36:1 class imbalance using geometric augmentation.

Deferred re-weighting (DRW), introduced in LDAM-DRW by Cao et al., provides a

relevant alternative by introducing class-balanced weights after initial representation learning. However, none of the reviewed retinal studies applies scheduled re-weighting, resampling schedules or long-tail-specific losses such as LDAM and logit adjustment.

Evidence remains limited because imbalance-handling methods are rarely evaluated through isolated ablations. Furthermore, Bano et al. (2025), Mane and Shekapure, and Kadhim et al. report only aggregate metrics, making minority-class improvements difficult to verify.

2.2.4 Vessel-based guidance

The retinal vasculature is visible in every fundus image and is altered by many diseases, which makes it a natural, disease-agnostic anatomical prior. The reviewed work uses it in three ways, summarised in Table 2.3.

Table 2.3: Use of vessel information in the reviewed studies

Study	Role of vessels	Method	Reported result
Bano et al. (2025)	Vessel and lesion maps fed to a downstream classifier (two-stage pipeline)	Attention U-Net (DRIVE) then CNN (LVPEI, 5-class DR)	Acc. 90%, precision 87%, recall 89%, F1 88%
Kadhim et al. (2024)	Hand-crafted vessel features concatenated with CLAHE features	Classical top-hat, homomorphic and matched-filter segmentation; CNN (5-class DR)	96.7% vs 72 to 88% for single-feature or raw-image baselines
Wang et al. (2024)	Segmentation only	Semi-dilation supervision (dilates only below-average-radius vessels) and a boundary reconstruction network	DRIVE: SEN 84.65%, ACC 96.9%, AUC 98.84%
HRM-UNet	Segmentation only (candidate frozen prior)	VM-UNet with Haar wavelet down-sampling and residual paths	DRIVE: Acc. 0.9529, Se 0.7707, F1 0.7988

ELA-U-Net	Segmentation only (lightweight alternative)	ELA with a dilated-convolution U-Net, 0.34M parameters	DRIVE: Acc. 0.9644, Se 0.8139, F1 0.8276

Bano et al. provide the closest precedent to a vessel-guided classifier: a segmentation network trained on DRIVE produces vessel maps that a separate CNN classifies into five DR grades. Both stages are trained once and fixed, the dataset is small and lopsided (226 No-DR images against 50 for each other grade), and results are aggregate only. Kadhim et al. offer empirical support for the premise that vessel structure carries information a plain CNN does not fully exploit, although their extractor is classical and their task is single-disease grading. Wang et al. contribute a principled treatment of vessel scale, recognising that thin and thick vessels need different supervision, and thereby a template for a structure-preserving vessel signal, but stay within segmentation. Among segmenters, HRM-UNet is the strongest in fine-capillary preservation, which matters because microvascular change is diagnostically relevant, while ELA-U-Net achieves higher headline accuracy with far fewer parameters. A semi-supervised segmentation approach that estimates uncertainty adaptively (AUE, reported at a Dice score of 83.66% on STARE) is relevant when labelled vessel masks are scarce. Finally, the survey attributes a gain of about 2 to 5% to U-Net-style segmentation as preprocessing.

Two cautions emerge. First, a hybrid ResNet-UNet reported on APTOS 2019 (85.17% accuracy, AUC 0.946) fuses segmentation and classification through a dual-pathway attention of the form F_attended = F_fused multiplied by attention weights, but is restricted to DR. Second, because many retinal conditions, such as drusen and ARMD, are not primarily vascular, a method that forces the network to attend to vessels may hurt on those diseases. None of the reviewed studies provides a mechanism that lets the classifier choose how much to use the vessel prior, and an offline check that a segmenter attends to real vasculature, and not to structures such as the optic disc, has been raised as a necessary precaution.

2.2.5 Curriculum learning

Curriculum learning orders or weights training examples from easy to hard, so that a model first learns stable general features and is exposed to difficult examples progressively. Self-paced variants derive difficulty from the model's own loss. In the long-tail setting, class-level analogues such as deferred re-weighting delay the emphasis on rare classes until representations are established. None of the reviewed retinal classification studies implements a curriculum. The nearest relative is the semi-dilation strategy of Wang et al., which treats vessels differently according to a difficulty-like criterion (radius relative to the mean), but this operates at the pixel level inside segmentation and not at the sample or class level inside classification. In particular, no reviewed work orders samples or classes using a signal derived from vessel segmentation, such as the confidence of a vessel prior or the agreement between the model's attention and the vessels.

2.2.6 Uncertainty estimation and calibration

A classifier used for triage must output probabilities that match observed frequencies and should indicate when it is unsure. The reviewed literature barely addresses this. CLARE-Net is the only fundus-classification study that reports ECE (0.0649) and a reliability diagram, but it gives no baseline for comparison and does not specify a calibration technique, so the improvement attributable to calibration is unknown. Bano et al. show a per-class confidence score in a demonstration interface, but this is the raw softmax output and is described as clinically meaningful without any calibration procedure. Mane and Shekapure cite a nine-model Bayesian neural network ensemble that estimates DR uncertainty and defers low-confidence predictions for human review, but do not implement it. In segmentation, a semi-supervised method (AUE) uses a dynamically updated, uncertainty-based threshold during training, which is conceptually close to uncertainty-aware triage.

Standard post-hoc remedies include temperature scaling, which rescales logits by a single learned scalar and is simple and cheap, and richer methods such as Dirichlet calibration, which can correct class-wise miscalibration; the latter is relevant to the tail,

where miscalibration is expected to be worst. Monte Carlo (MC) dropout offers a practical way to obtain a spread of predictions from a single trained network. For rare classes there is a further caveat: because most multi-label predictions are true negatives near zero, aggregate calibration measures can look good while tail-class calibration is poor, so equal-mass binning and class-wise reporting are advisable.

2.2.7 Explainability

Explainability is common but qualitative. Grad-CAM, Grad-CAM++, SHAP, LIME, RISE and Integrated Gradients appear in the studies of Sekhar et al., Venkatappa et al. and Anusha et al., usually as overlays on one or two sample images. In Venkatappa et al. each method was applied to a different network, so the methods cannot be compared with each other. No study quantifies agreement between the explanation and an anatomical reference, such as overlap with annotated lesions or with the vessels, so there is no evidence on whether the highlighted regions are the clinically relevant ones.

2.2.8 Multi-disease classification and limited data

A multi-source knowledge fusion ensemble for OCT classification addresses four distinct pathologies (choroidal neovascularisation, diabetic macular oedema, drusen and normal) with only 500 training images. It concatenates features from three frozen ImageNet-pretrained backbones (ResNet18, DenseNet121 and InceptionV3) and trains only a lightweight head. This is a genuine multi-disease precedent and shows that frozen pretrained features are a practical response to data scarcity, which is related to, but distinct from, long-tail imbalance. It uses no vessel information, no curriculum and no calibration, applies standard cross-entropy with single-pass softmax predictions, and reports no per-class metrics.

2.2.9 Evaluation practice

Reading the studies together reveals recurring weaknesses in evaluation. Test sets are small (165 images in Venkatappa et al.) and rare classes may have two or three test samples; no study reports confidence intervals or results over multiple seeds; none evaluates on an external dataset; and RFMiD itself is redefined differently from paper to

paper (8, 10 or 46 classes; single-label or multi-label), which prevents direct comparison. Metric choice can also mislead: in Venkatappa et al., DenseNet-121 reaches 88.48% accuracy but a macro-F1 of 0.697, in Sekhar et al., 82.26% accuracy sits beside a macro-F1 of about 0.60, and per-class accuracy in a sparse multi-label problem (Anusha et al.) is inflated by negatives. The survey itself recommends macro-averaged and class-wise metrics in place of aggregate accuracy. A two-tier design such as that of Anusha et al. also propagates errors, since a diseased image classified as healthy by the first tier never reaches the second; 25 of 490 diseased validation images (about 5%) fall into this category.

2.2.10 Summary of coverage

Table 2.4 summarises which of the components central to this project are present in each reviewed study.

Table 2.4: Coverage of the key components across the reviewed studies

Study	Vessel prior	Curriculum	Calibration	Per-class reporting	Multi-label
CLARE-Net	No	No	Partial (ECE, no baseline)	Yes (sensitivity, specificity)	No
Sekhar et al.	No	No	No	Partial (macro-F1)	No
Venkatappa et al.	No	No	No	Yes (very small n)	No
Anusha et al.	No	No	No	Inflated metric	Yes
Bano et al.	Yes (input to CNN, fixed)	No	No (raw softmax)	No	No
Kadhim et al.	Partial (classical features)	No	No	No	No
Mane and Shekapure	No	No	Cited only	No	No

Wang et al.	Segmentation only	Implicit, pixel-level	No	Not applicable	Not applicable
OCT fusion ensemble	No (OCT)	No	No	No	No (4-way multi-disease)
This work	Yes (flexible gates)	Yes (two-level soft)	Yes (class-wise, MC dropout)	Yes (with bootstrap CIs)	Yes

2.3 RESEARCH GAPS

The survey above shows that individual components of the problem have been studied, mostly in isolation and with limited rigour. Seven gaps follow from it. For each, the evidence from the literature and the response taken in this project are stated. Whether each response actually improves performance is an empirical question, and the outcomes, including those that are negative, are reported in Chapter 5.

Gap 1: No curriculum for long-tailed retinal classification

Every reviewed study that addresses imbalance does so with static weighting, thresholds, loss re-shaping or augmentation. This project introduces a two-level soft curriculum that combines class difficulty (rarity and label cardinality) with sample difficulty (loss history and vessel misalignment) into a pacing weight on the per-sample loss, and compares it against static weighting.

Gap 2: Vessel structure is not used flexibly inside a multi-disease classifier

Where vessel information is used (Bano et al.; Kadhim et al.), it is an input-level signal for single-disease DR grading, computed once and fixed. The multi-label studies use no vessel information, and none lets the classifier decide how much to rely on vessels, although many conditions are not vascular. The vessel prior has also never been connected to the training schedule. This project injects a frozen HRM-UNet vessel map into DenseNet-121 through learnable, zero-initialised gates at intermediate scales, so that the contribution of the prior is learned per scale and can be suppressed, and it also uses vessel agreement as one of the curriculum difficulty signals.

Gap 3: Confidence is reported but not calibrated, and rarely used for triage

Only one reviewed study reports ECE, with no uncalibrated baseline and no named method; the others show raw softmax confidence or cite uncertainty methods without implementing them. Class-wise calibration and uncertainty-based deferral are absent. This project applies a post-hoc calibration chain (temperature scaling followed by class-wise scaling, fitted without leakage on out-of-fold predictions), reports ECE with equal-mass bins before and after calibration, and evaluates MC-dropout uncertainty as a triage signal.

Gap 4: Explanations are qualitative

Gradient-based and perturbation-based explanations are presented as sample overlays, with no quantitative check against anatomy. This project computes a correlation between Grad-CAM maps and the vessel map, and examines the learned gate parameters, to test directly whether vessel guidance changes where the model looks.

Gap 5: Evaluation is statistically weak and often misleading

Small test sets, rare classes with two or three test samples, no confidence intervals, single runs, and metrics inflated by true negatives limit what the published comparisons can establish. This project pools the official training and validation images into a 2,560-image set evaluated by five-fold multilabel-stratified cross-validation, holds the official 640-image test set out, uses paired bootstrap comparisons, and reports macro-level and per-class metrics together with a tail-focused breakdown, so that differences can be judged against their uncertainty.

Gap 6: Inconsistent label and split definitions

RFMiD is used with 8, 10 or 46 classes and as single-label or multi-label, so published numbers cannot be compared. This project uses a stated and reproducible protocol: all 25 labels with at least ten training positives are retained, the task remains multi-label, and the official split is respected.

Gap 7: No external validation

None of the reviewed studies validates on an independent dataset, and the survey reports

a 10 to 20% loss of accuracy under domain shift. This gap is not closed by the present work, which uses RFMiD only; cross-dataset evaluation is identified as future work, and the results should be read as in-distribution.

Table 2.5: Research gaps and the corresponding response in this work

Gap identified in the literature	Response in this work
1. No curriculum scheduling for the long tail; static weighting only	Two-level soft curriculum with a pacing weight on the per-sample loss
2. Vessel prior not used flexibly, fixed input-level use only, never linked to training	Frozen HRM-UNet map injected through learnable gates in DenseNet-121; vessel agreement used as a curriculum signal
3. Confidence uncalibrated or calibrated without a baseline; no class-wise calibration or triage	Temperature and class-wise scaling fitted on out-of-fold logits; ECE before and after; MC-dropout triage
4. Qualitative explanations only	Grad-CAM to vessel-map correlation and gate analysis
5. Small test sets, no confidence intervals, misleading metrics	Five-fold cross-validation, held-out test set, paired bootstrap, macro and per-class reporting
6. Inconsistent RFMiD label definitions	Explicit 25-label multi-label protocol with the official split
7. No external validation	Not addressed; listed as a limitation and future work

In summary, the literature establishes strong baselines (DenseNet-121 and EfficientNet backbones, augmentation and weighted losses) and demonstrates the promise of vessel information for DR, but it does not combine vessel guidance, curriculum scheduling, calibrated uncertainty and rigorous tail-aware evaluation for multi-disease classification. The next chapter describes the system designed to examine these components together.

CHAPTER 3
PROPOSED SYSTEM AND METHODOLOGY

3.1 OVERVIEW

Retinal fundus photography is the principal low-cost modality for screening vision-threatening conditions such as diabetic retinopathy, age-related macular degeneration and retinal vein occlusion. Automating multi-disease screening is difficult for three reasons that this work addresses directly. First, the label distribution is severely long-tailed: in the Retinal Fundus Multi-disease Image Dataset (RFMiD) the most common retained class outnumbers the rarest by 138:1, so a classifier can score well on aggregate metrics while failing on the rare diseases that matter clinically. Second, disease evidence is partly vascular and partly non-vascular, so a model should be able to exploit vessel structure without being forced to depend on it. Third, the predicted probabilities of a deep network are rarely trustworthy, which limits their value for triage and clinical decision support.

This chapter presents the proposed system, a vessel-guided multi-label classifier with calibrated and uncertainty-aware outputs, together with the methodology used to train and evaluate it. The system combines four ideas:

- **Vessel guidance.** A frozen vessel-segmentation network (HRM-UNet, pre-trained on DRIVE) produces a vessel probability map for every fundus image. The map modulates intermediate features of a DenseNet-121 classifier through learnable, residual gates at three scales.
- **Long-tail handling.** Per-label binary cross-entropy with capped inverse-frequency positive weights and deferred re-weighting, with Asymmetric Loss, mixup and stochastic weight averaging evaluated as additional training recipes.
- **Two-level curriculum learning.** A training-time scheduler that ranks samples by class difficulty and sample difficulty and applies a soft, per-sample loss weight, evaluated as an ablation.
- **Uncertainty calibration.** Post-hoc temperature scaling followed by class-wise

scaling, fitted on out-of-fold predictions, and Monte-Carlo dropout to quantify predictive uncertainty for triage.

The methodology places equal weight on measurement. Because most tail classes contain only a handful of test positives, single-split comparisons cannot separate genuine improvements from noise. The evaluation protocol therefore uses multilabel-stratified five-fold cross-validation over the pooled training and validation data, keeps the official test split strictly held out, and compares recipes with a paired bootstrap. Table 3.1 summarises the processing stages of the proposed system.

Table 3.1: Stages of the proposed system

Stage	**Module**	**Trainable**
0	Preprocessing: field-of-view (FOV) crop, CLAHE, resizing	No
1a	Vessel prior: HRM-UNet (hrm_unet_drive.pth)	Frozen
1b	Backbone: DenseNet-121 (ImageNet pre-trained); RETFound ViT-L/16 evaluated as a trial	Yes
2	Vessel-guided attention fusion at three scales	Yes
3	Classification head: global average pooling, dropout, one sigmoid output per retained class	Yes
4	Post-hoc calibration: temperature scaling followed by class-wise scaling, fitted on out-of-fold predictions	Post-hoc
5	Training-time recipes: curriculum scheduler, Asymmetric Loss, mixup, SWA	Training only

3.2 SYSTEM ARCHITECTURE

3.2.1 Overall Architecture

The system is organised as a pipeline in which a single fundus image is converted into two spatially aligned views, one for the classifier and one for the vessel segmenter. The segmenter output is passed to fusion gates inside the classifier, and the resulting logits are calibrated before being converted into per-class probabilities and an uncertainty estimate. The curriculum scheduler operates only during training, by re-weighting the loss, and plays no part in inference. Figure 3.1 shows the complete architecture.

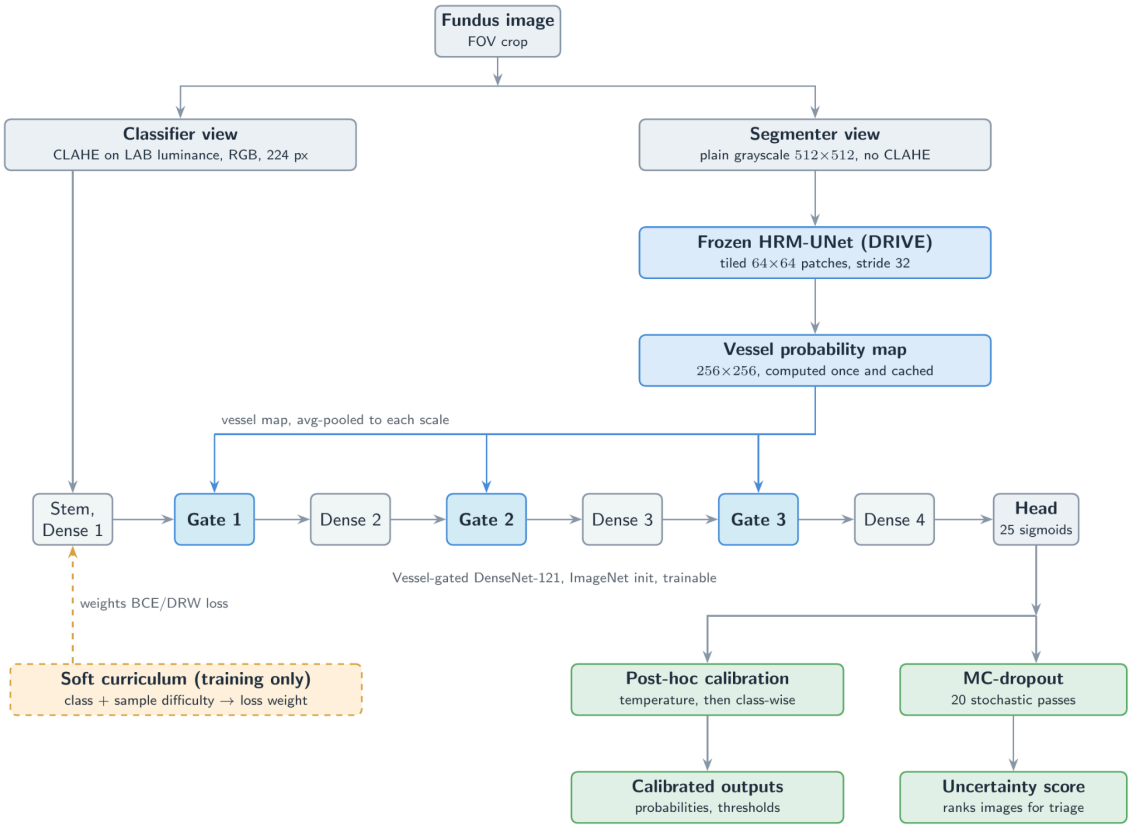


Figure 3.1: Overall architecture of the proposed vessel-guided classification system

3.2.2 Preprocessing Module

Each image is decoded once and cropped to the retinal field of view by thresholding intensity (values below 10 are treated as background). This step matters in its own right: roughly 30% of raw pixels are background, so cropping lets the retina fill the frame. Two aligned views are then derived from the same crop so that the vessel map remains pixel-registered with the features it modulates:

- **Classifier view.** Contrast-limited adaptive histogram equalisation (CLAHE, clip limit 2.0, 8×8 tiles) applied to the luminance channel of the LAB colour space and resized to a square RGB image. Equalising luminance only improves contrast without shifting hue, which matters for haemorrhage and exudate classes.
- **Segmenter view.** Plain grayscale at 512×512 pixels with no CLAHE, matching exactly how HRM-UNet was trained on DRIVE.

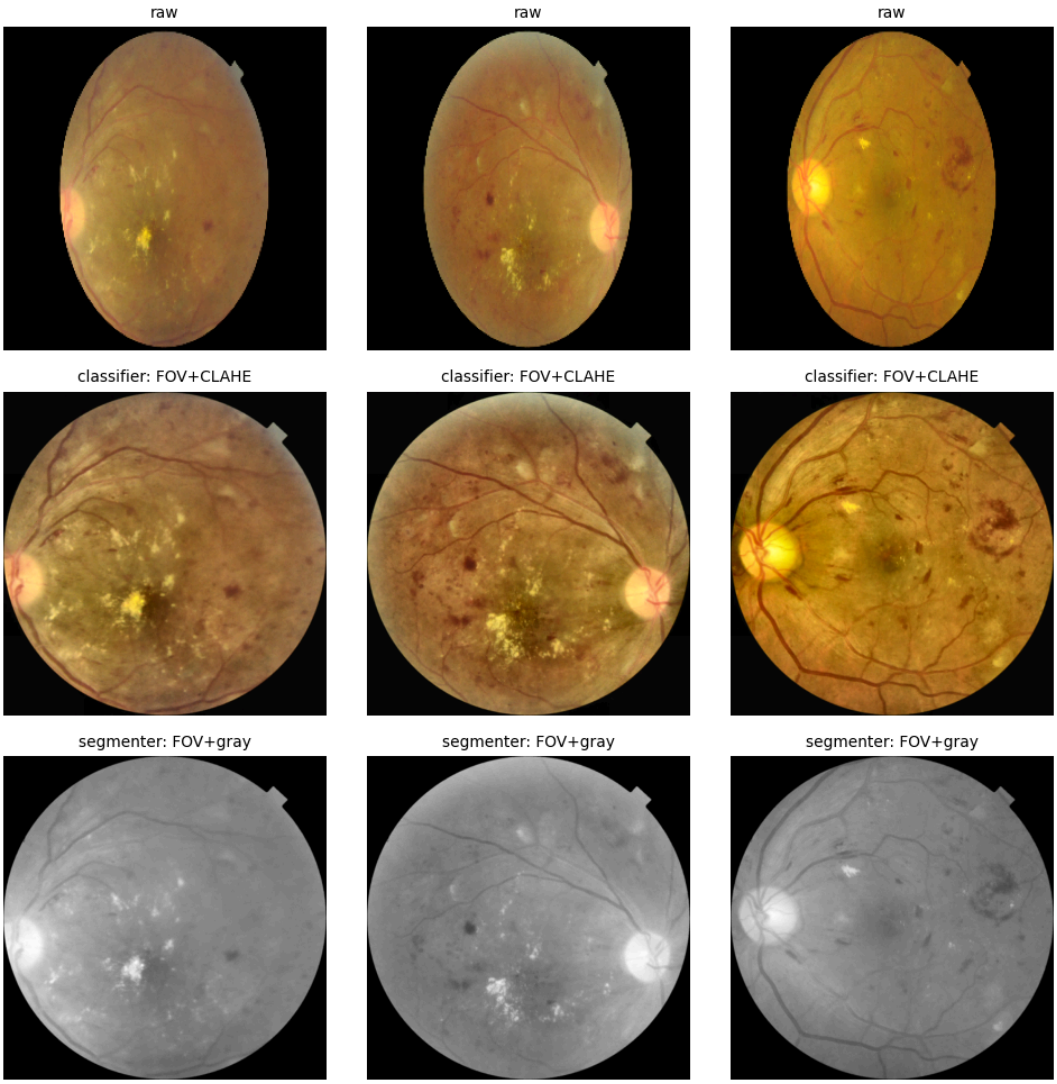


Figure 3.2: Preprocessing pipeline and the two aligned views

3.2.3 Frozen Vessel Prior (HRM-UNet)

The vessel prior is an HRM-UNet: a patch-embedding layer, four Visual State Space (VSS) stages built on Mamba-style selective-scan (SS2D) blocks with Haar-wavelet downsampling, residual-path skip connections and a patch-expanding decoder. The network is trained on DRIVE and kept frozen and in evaluation mode, so its weights are never altered by classification training. Because the segmenter is frozen, its outputs are deterministic and are computed once and cached, which is exactly equivalent to recomputing them at every epoch.

Three settings are inherited from the segmenter's training rather than chosen for this task: the input is grayscale scaled to [0, 1] without mean/standard-deviation normalisation; no contrast enhancement is applied; and inference is tiled (64×64 patches, stride 32) rather than full-image. Model hyper-parameters are recovered automatically from the checkpoint tensor shapes, and weights are loaded strictly so that any mismatch raises an error instead of silently leaving layers randomly initialised.

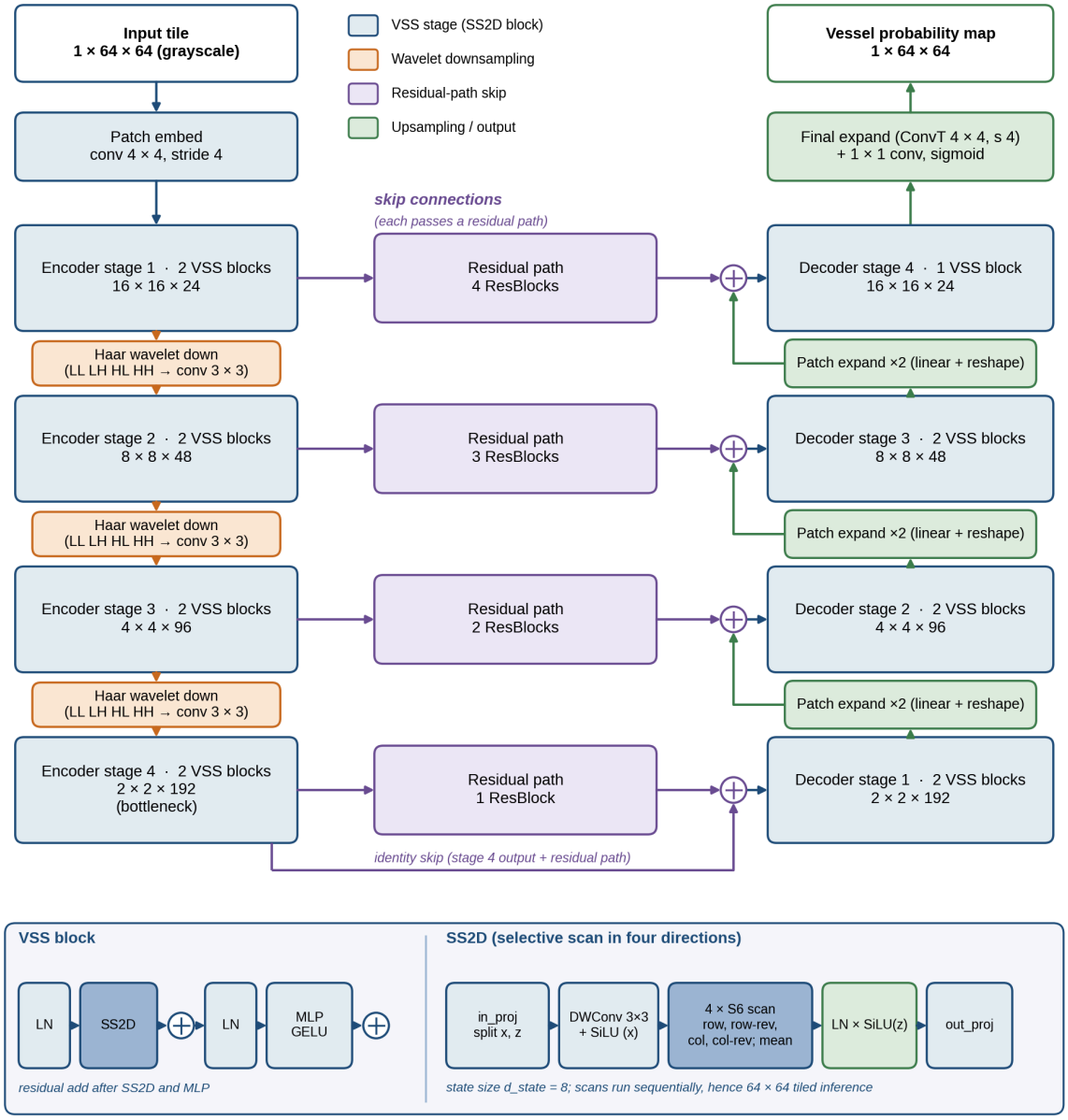


Figure 3.3: HRM-UNet vessel segmentation architecture

3.2.4 Vessel-Guided Classifier

The classifier is a DenseNet-121 initialised from ImageNet weights. The vessel map is injected after the first three transition layers, where feature maps have 128, 256 and 512 channels. At a 224×224 input these maps are 28×28, 14×14 and 7×7 respectively, and the spatial size scales with the chosen input resolution. At each injection point a vessel gate computes an attention tensor from the vessel map and applies it to the features through a residual connection:

$$A = \sigma(MLP_{1\times1}(AvgPool(V))) \qquad F_{out} = F \cdot (1 + \gamma \cdot A)$$

where V is the vessel probability map, average-pooled to the spatial size of the feature map F, and the 1×1 MLP maps one channel to a hidden width of 16 and then to the channel count of F. The scalar γ is learnable and initialised to zero, with the final bias initialised to zero, so that at the start of training the network is exactly the pretrained backbone and vessel guidance is learned as a departure from a known-good baseline. The residual form also allows the network to down-weight vessel information, which is important because roughly half of the RFMiD conditions are non-vascular.

Features from the final dense block are batch-normalised, passed through ReLU and global average pooling, then through dropout (p = 0.3) and a single linear layer with one logit per retained class. Outputs are independent sigmoids rather than a softmax, because a fundus image may carry several concurrent diseases. The same gate module is reused in a RETFound ViT-L/16 trial, applied to the 14×14 patch-token grid after transformer blocks 8, 16 and 24.

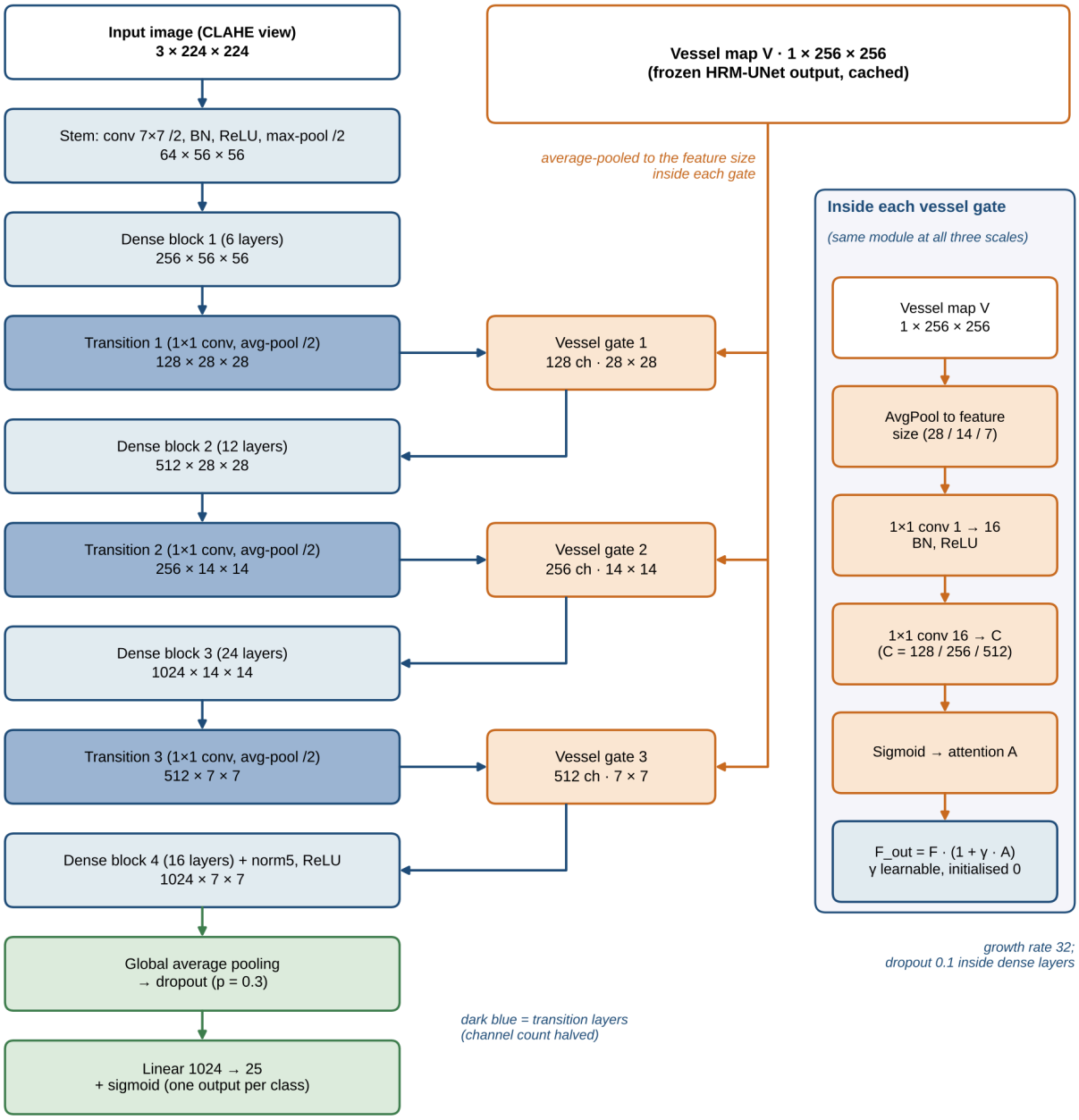


Figure 3.4: Multi-scale vessel-gated DenseNet-121

3.2.5 Long-Tail Loss and Curriculum Scheduler

The training objective is a per-label binary cross-entropy with logits. Class imbalance is handled by a capped inverse-frequency positive weight (cap = 20) computed separately for each fold from that fold's training labels, so the held-out fold never influences the

loss weighting. Deferred re-weighting keeps the weights at one for the first half of training and then ramps them up, allowing representations to form before the loss is tilted towards rare classes. The loss is retained per sample (no reduction), which gives the curriculum its difficulty signal at no additional cost.

The two-level curriculum scheduler expresses sample difficulty as two separately computed levels that are then combined. All signals are converted to percentile ranks so that their different raw scales cannot dominate the ordering.

Table 3.2: Difficulty signals of the two-level curriculum

Level	Signal	Weight	Source
Class difficulty	Rarity of the sample's rarest positive label	0.15	Static
Class difficulty	Label cardinality (number of positive labels)	0.15	Static
Sample difficulty	Exponential moving average of per-sample loss (EMA 0.7)	0.55	From training loss
Sample difficulty	Vessel misalignment = 1 − vessel confidence	0.15	One forward pass per refresh

Vessel confidence is the per-sample correlation between the model's gate attention and the vessel prior, so a sample whose attention agrees with the true vasculature is treated as easier. Samples without any disease label (about 21% of RFMiD) receive median rather than zero rarity; scoring them zero filled the warm-up pool with empty images and collapsed the model to all-negative output. Pacing is implemented as a soft weight, not a hard-excluded pool: a sigmoid centred on the pace threshold gives samples easier than the pace a weight close to one, and harder samples a weight that decays smoothly towards a floor of 0.1, so every sample contributes gradient in every epoch. The pace starts at 0.60, reaches 1.0 at 75% of training, and the warm-up stage occupies the first 25% of epochs.

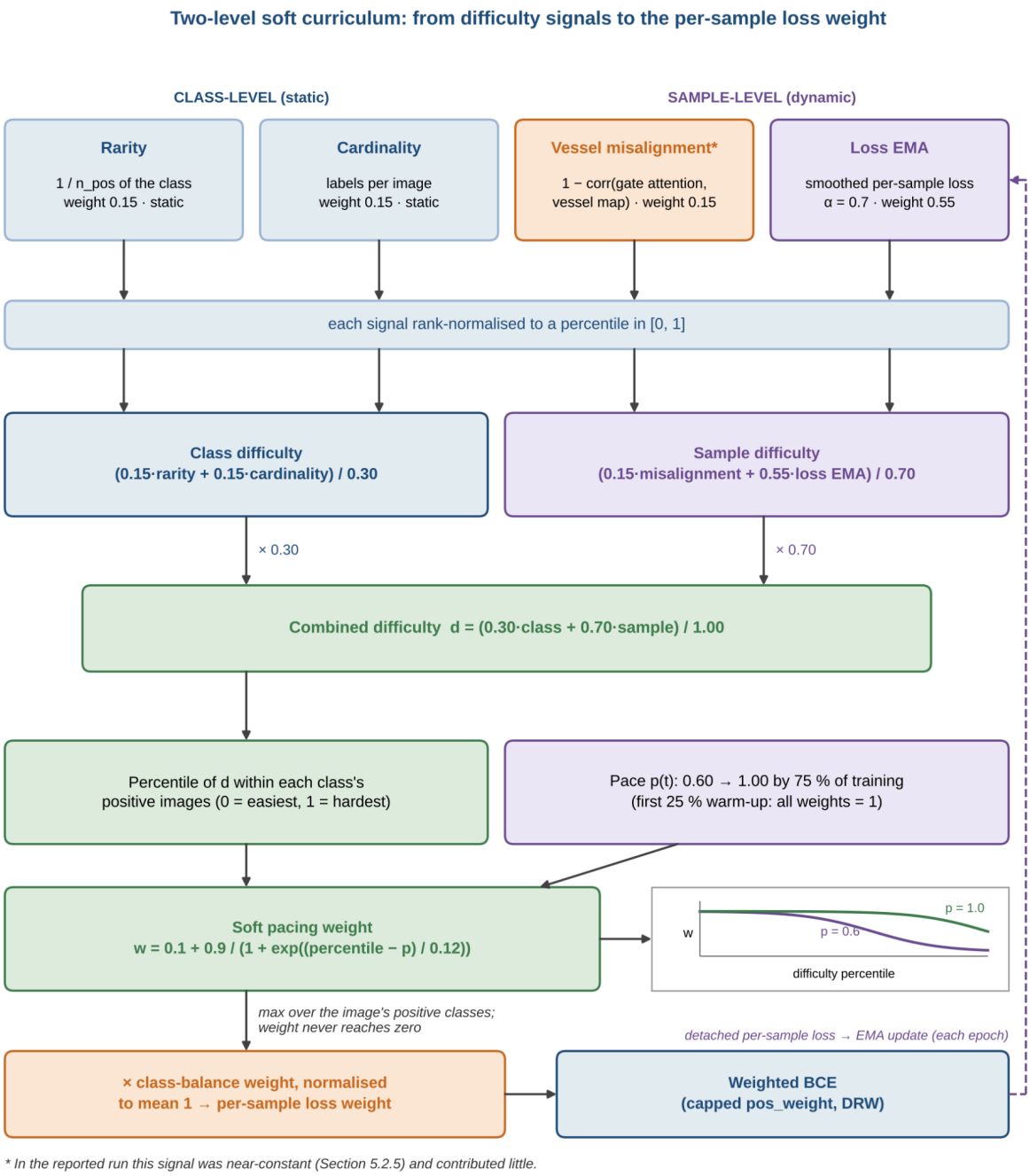


Figure 3.5: Two-level curriculum scheduler

3.2.6 Calibration and Uncertainty Module

Calibration is applied after training and is rank-preserving within each class, so it changes probability quality but not the ranking-based metrics. It is chained in two steps.

Temperature scaling learns a single scalar T that rescales all logits. Class-wise scaling (a diagonal-Dirichlet form) then learns a per-class scale and offset, which is needed because miscalibration differs sharply between common and rare classes. Both calibrators are fitted on out-of-fold logits rather than on a small validation split. Predictive uncertainty is obtained by Monte-Carlo dropout: dropout in the classification head is kept active at inference, batch normalisation stays in evaluation mode, and 20 stochastic forward passes give a per-class mean and standard deviation. The standard deviation ranks images by uncertainty for triage.

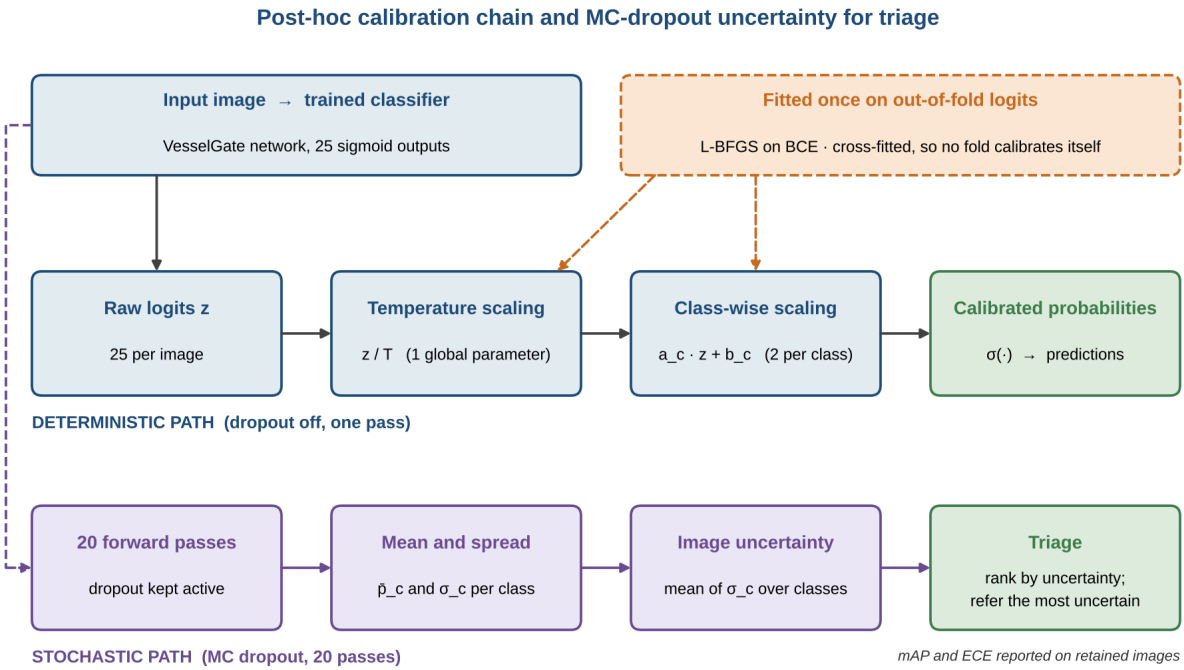


Figure 3.6: Calibration and uncertainty pipeline

3.2.7 Inference Pipeline

At inference the path is: preprocess → vessel map from the frozen HRM-UNet → vessel-gated DenseNet-121 → logits → calibrated probabilities → per-class thresholds. For the reported test results, the logits of the five fold models are averaged into an ensemble before calibration. Per-class decision thresholds are chosen by maximising F1 on out-of-fold predictions, never on test data.

3.3 METHODOLOGY

3.3.1 Dataset and Label Preparation

Experiments use RFMiD, which provides 1,920 training, 640 validation and 640 test fundus images, each annotated with a Disease_Risk (normal versus abnormal) label and 45 disease labels, for 46 labels in total. On average each image carries about 1.9 positive labels, confirming the multi-label nature of the task. Classes with fewer than 10 positives in the original training split are removed immediately after loading, which reduces the label set from 46 to 25 evaluable classes (21 classes dropped). This filtering is a measurement choice and not a modelling improvement; it makes the evaluation meaningful and keeps results comparable across versions of the pipeline. The 25 retained classes are grouped by prevalence in the original training split into 8 head classes (at least 5%), 8 tail classes (at most 1%) and 9 mid-frequency classes. The imbalance ratio between the most and least frequent retained classes is 138:1.

Table 3.3: Dataset summary after preprocessing of labels

Item	Value
Original splits (train / validation / test)	1,920 / 640 / 640 images
Labels as shipped	46 (Disease_Risk + 45 diseases)
Classes retained (≥ 10 training positives)	25 (8 head, 9 mid, 8 tail)
Development pool (train + validation)	2,560 images
Held-out test set	640 images (never used for selection, calibration or thresholds)
Imbalance ratio (max : min)	138 : 1

3.3.2 Evaluation Protocol

Earlier comparisons on a single validation split and 640 test images were unreliable: most tail classes have only four to five test positives, and the observed vessel-guidance gain (+0.012 mAP) was smaller than the seed-to-seed spread. The protocol was therefore redesigned as follows.

- **Pooling and hold-out.** Training and validation images are pooled (2,560 images). The 640-image test split remains untouched until final scoring.

- **Stratified five-fold cross-validation.** Folds are built with a compact iterative-stratification procedure for multilabel data (after Sechidis et al., 2011): samples are visited rarest-label first and assigned to the fold that still needs the most of that label. Every class is spread within ±1 positive across folds, which is asserted in code.
- **Out-of-fold (OOF) prediction.** For each fold the model trains on four-fifths of the pool, selects the best epoch on the held-out fifth, and predicts both that fifth and the test split. Pooled OOF predictions cover all 2,560 images, each predicted by a model that never saw it, giving roughly four times more positives per tail class than the single test split.
- **Paired bootstrap.** OOF images are resampled and the mAP of two recipes is recomputed on the same resample (300 resamples). A recipe is declared better or worse only if the 95% confidence interval of the difference excludes zero.

3.3.3 Preprocessing and Caching

Preprocessing is executed once and the results are stored as arrays. Classifier-view images are cached at 448×448 pixels so that the resolution sweep (224, 320, 384 pixels) requires no rebuild, and vessel maps are cached at 256×256 because they are smooth probability maps that the gates average-pool to at most 48×48 anyway; they are resized on the GPU when needed. Cache construction uses a thread pool with thread-local CLAHE objects, because OpenCV CLAHE instances are not thread-safe, and is idempotent and resumable so that long sessions can survive interruption. Table 3.4 lists the cache contents.

Table 3.4: Cached arrays

Array	Shape	Approx. size
Pool images	(2560, 448, 448, 3) uint8	1.5 GB
Pool vessel maps	(2560, 256, 256) uint8	170 MB
Test images and vessel maps	(640, …)	0.4 GB

3.3.4 Vessel Segmentation Inference

The segmenter's state-space core scans sequentially, so its cost grows with sequence length: 4,416 steps for a 64×64 patch versus 282,624 for a 512×512 image. Tiled inference is therefore both faithful to training and much faster. Each grayscale image is resized to 512×512 so that vessel width in pixels stays close to that of DRIVE, divided into 64×64 patches at stride 32 (50% overlap), passed through the network in large batches of patches, passed through a sigmoid, and re-assembled by averaging overlapping predictions using unfold and fold operations. The probability map is then downsampled to 256×256 for caching. Before the full cache is built, the segmenter is checked on a few samples to confirm that its output follows real vasculature.

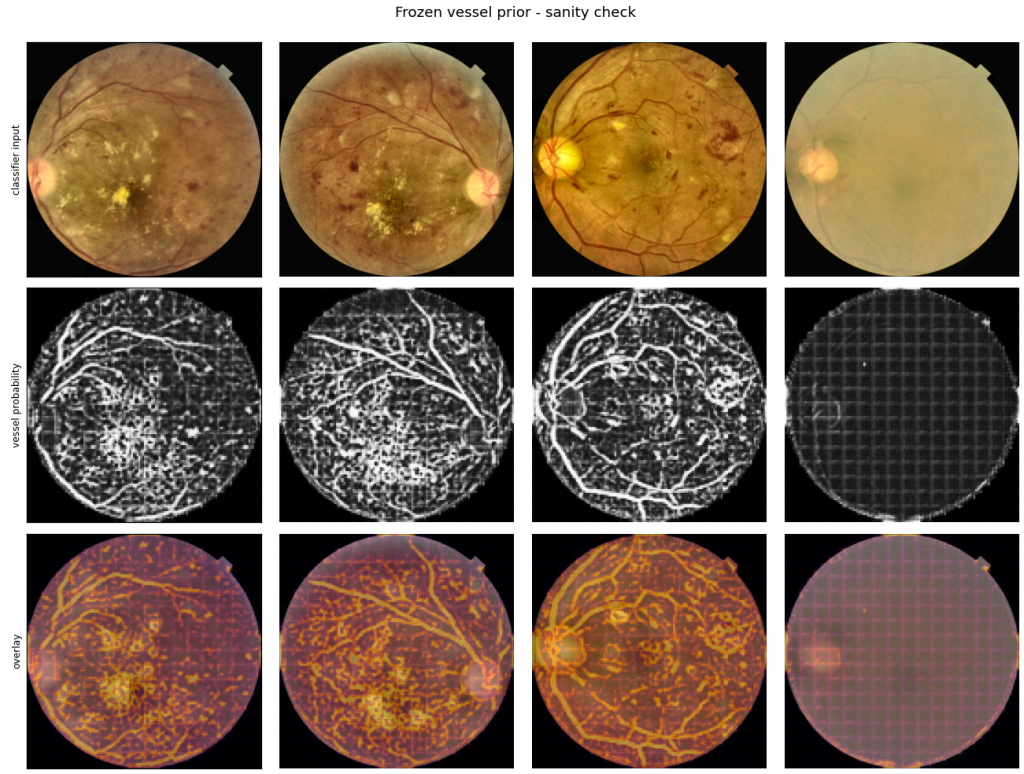


Figure 3.7: Example vessel maps produced by the frozen HRM-UNet

3.3.5 Data Loading and Augmentation

Augmentation runs on the GPU. A single affine grid and grid-sample operation applies

per-sample rotation (±15°), zoom, shift and horizontal flip to the image and vessel map stacked as one four-channel tensor. Joint geometry is a correctness requirement, because the fusion gates multiply the vessel map into features at matching positions. The two caches have different sizes, so both are first brought to a canvas of input size × 256/224, using area interpolation for the image and bilinear interpolation for the vessel map. Images are then centre-cropped to the input size and normalised with ImageNet statistics.

A stronger, lesion-preserving photometric regime is applied to the RGB image only (the vessel map is never perturbed): per-channel gain of ±6%, saturation change of ±15%, mild blur on 20% of samples and sensor noise up to $\sigma = 0.02$. Cutout and random erasing are deliberately not used because they can erase the single lesion that defines a rare class, and vertical flipping is avoided because fundus orientation is anatomically consistent.

3.3.6 Model Training

The baseline training regime was designed to reduce the generalisation gap observed in earlier experiments, where training loss fell to about 0.03 while validation mAP plateaued and test mAP was markedly lower. Key settings are listed in Table 3.5.

Table 3.5: Training configuration

Component	Setting
Backbone / head	DenseNet-121 (ImageNet), dropout 0.3 in head, 0.1 in dense layers
Optimiser	AdamW, decoupled weight decay 0.05 (none on biases and normalisation scales)
Learning rates	Backbone top stage 1e-4; gates and head 1e-3 (10×)
Layer-wise LR decay	0.8 per backbone stage below the top
Schedule	20 epochs, 1 warm-up epoch, cosine decay stepped per iteration
Batch size	32 (16 × 2 gradient accumulation above 320 px input)
Precision	Automatic mixed precision with channels-last; loss computed in fp32
Gradient clipping	5.0
Model selection	Best validation-fold mAP epoch (never the test split)

Early stopping	Implemented but off by default, since validation mAP is noisy (±0.015)

Three additional training-time recipes from the literature on imbalanced multi-label learning are available. Asymmetric Loss (Ridnik et al., 2021) uses focusing parameters $\gamma^+ = 1$ and $\gamma^- = 4$ with a probability margin of 0.05 to down-weight easy negatives. Mixup draws a mixing coefficient from Beta(0.2, 0.2) and is applied to half of the batches. Stochastic weight averaging averages weights over the last 25% of training. A further option resamples images with replacement in proportion to class-rarity weights. Each is switched on or off through a single configuration object, so that one training run is fully defined by one record.

3.3.7 Curriculum Training Procedure

When the curriculum is enabled, difficulty scores are recomputed each epoch from the loss exponential moving average and the vessel-confidence signal. Class-balance weights are applied continuously from the first epoch. The product of the soft pacing weight and the class-balance weight is passed to the loss as a per-sample weight. Because nothing is excluded, every epoch visits all training samples exactly once, so the curriculum and non-curriculum arms receive identical numbers of gradient steps. Diagnostics confirm that the mean weight of every class stays above the floor at epoch 0, i.e. that the soft curriculum has not degenerated into hard exclusion.

3.3.8 Calibration Procedure

After the five fold models are trained, their OOF logits are pooled and used to fit the calibrators. To obtain an honest calibrated error estimate, calibration is cross-fitted: the calibrators applied to fold k are fitted on the other four folds. Test-split scores are produced for each fold model and for their ensemble, using calibrators and F1 thresholds fitted on the pooled OOF predictions. Because the best epoch is chosen on each fold's held-out data, OOF logits are mildly optimistic, so test-split numbers, which are never used for selection, are treated as the unbiased ones and both are reported.

3.3.9 Experimental Design

The contribution of each component is isolated by training named recipes under identical folds, seeds and data. Each recipe is cached by a hash of its settings and fold, so interrupted runs resume without repeating completed work.

Table 3.6: Experimental stages and research questions

Stage	Recipes	Folds	Question addressed
1	A_v7_regime, A_baseline, B_vessel, C_vessel_curric	5	Does the regularisation regime help? Is vessel guidance beneficial? Does the curriculum help?
3a	B_vessel at 320 and 384 px	5	Does higher input resolution help?
3b	A_retfound, B_retfound	1 (trial)	Does a fundus-pretrained ViT-L beat DenseNet-121, with and without vessel gates?
4	E0 … E6, G_all_v7	5 (ablation uses 3)	Which individual training changes help or hurt when added one at a time?

Recipe A is the DenseNet-121 baseline, B adds vessel guidance, and C adds the curriculum on top of B. The resolution sweep tests whether 224 px is too coarse for microaneurysms and drusen. RETFound is a ViT-L/16 masked-autoencoder pretrained on about 900,000 colour fundus photographs; it is fine-tuned with layer-wise LR decay of 0.75, drop-path 0.2 and gradient checkpointing, and its fusion variant applies the same gates to the token grid. In Stage 4, a bundle of seven simultaneous training changes was previously found to reduce mAP; here each change (retuned ASL, ASL warm-up, milder mixup, rarity oversampling, gentler learning rates, longer schedule) is added alone to a vessel + ASL + mixup + SWA base recipe, and the sum is also run, so that interactions can be separated from individual effects.

3.3.10 Evaluation Metrics

Accuracy is uninformative here because an all-negative prediction scores above 95% on every label. The following metrics are reported instead.

- **Ranking metrics.** Macro mean average precision (mAP) over classes with at

least one positive, chosen because average precision suits rare-positive detection where ROC-AUC can flatter; and macro AUC for comparability with published work. The RFMiD challenge score (Disease_Risk AUC plus 45-label mAP) is also computed.

- **Long-tail metrics.** Head, mid and tail mAP, split at 5% and 1% prevalence, as the principal indicator of performance on rare diseases. Tail results are restricted to classes with at least five evaluation positives where noted.
- **Thresholded metrics.** Per-class F1 with thresholds chosen by maximising F1 on OOF predictions (grid of 49 values from 0.02 to 0.98), averaged over all classes (macro-F1) and over tail classes. Sensitivity, specificity and precision are also computed for the normal-versus-abnormal screening label and for diabetic retinopathy.
- **Calibration metrics.** Expected Calibration Error (ECE) with 15 equal-mass bins, Brier score and negative log-likelihood. Equal-width bins degenerate in this setting because most (sample, class) pairs are true negatives predicted near zero. Pooled ECE is also scale-blind to errors on rare classes, so ECE is reported separately for head, mid and tail groups.
- **Statistical comparison.** Paired-bootstrap differences in OOF mAP with 95% confidence intervals.

3.3.11 Interpretability and Uncertainty Analysis

Interpretability is assessed with Grad-CAM taken from the final normalisation layer. In addition to visual inspection, a quantitative agreement score is computed as the correlation between each class activation map and the frozen vessel prior. If vessel guidance is working as intended, agreement should be higher for vascular conditions (diabetic retinopathy, branch and central retinal vein occlusion) than for non-vascular ones (media haze, drusen); the absence of that difference would indicate that the model attends to vessels regardless of disease. The learned gate coefficients γ at each scale are inspected as a second check.

For uncertainty triage, 20 MC-dropout passes are run on the test split, images are

ordered by mean predictive standard deviation, and error rates are compared between the most and least uncertain fractions to test whether uncertainty identifies likely mistakes.

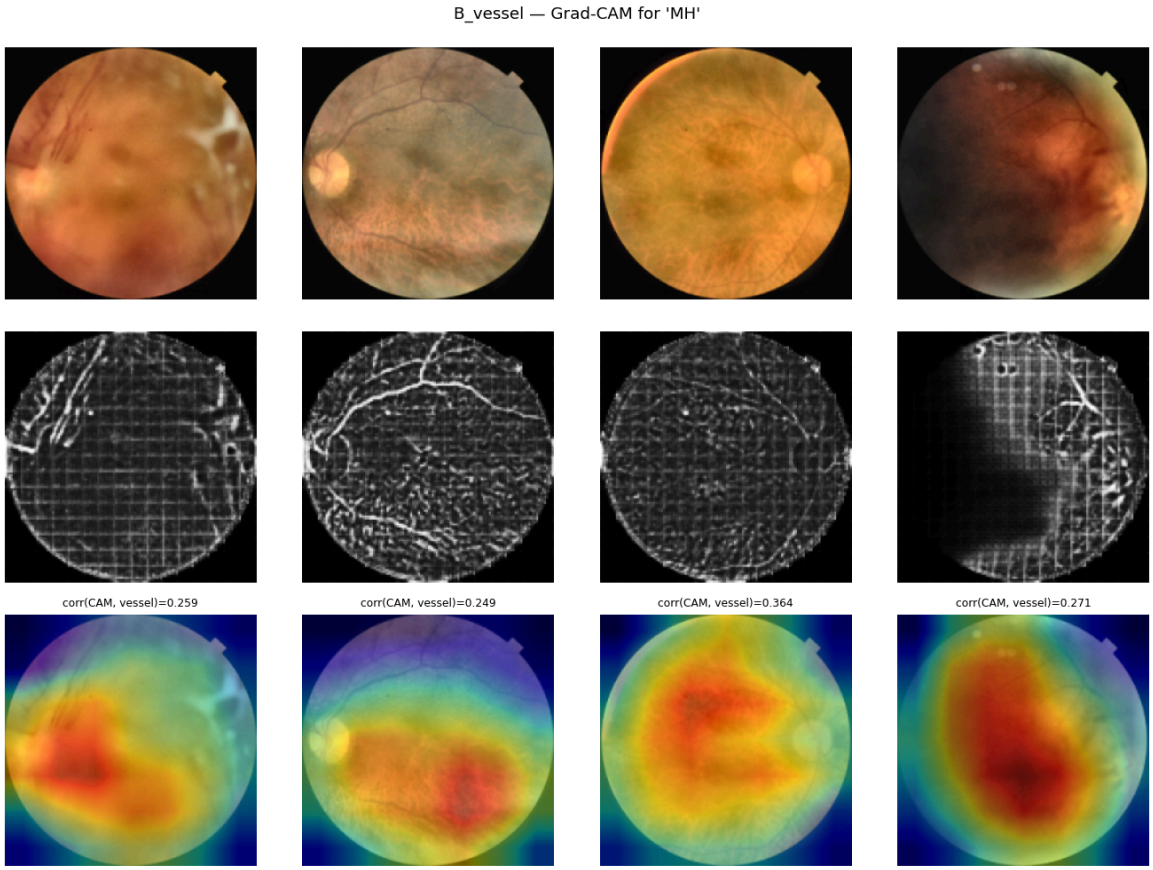


Figure 3.8: Grad-CAM overlays and vessel-agreement analysis

3.3.12 Implementation Details

The system is implemented in Python with PyTorch and torchvision (DenseNet-121), timm (ViT-L/16), OpenCV (preprocessing), scikit-learn (metrics) and pandas/NumPy (data handling), and was run on a single NVIDIA T4 GPU (15 GB) in a Kaggle environment. Random seeds are fixed for reproducibility, and the notebook includes a smoke-test mode that runs every stage for two epochs on one fold as an end-to-end check. All recipes, fold predictions and caches are persisted so that experiments are resumable and the exact configuration of any reported result can be recovered.

Table 3.7: Software and hardware environment

Item	Details
Language and framework	Python, PyTorch, torchvision, timm
Image and data libraries	OpenCV, NumPy, pandas, scikit-learn, matplotlib
Hardware	NVIDIA T4 GPU (15 GB), Kaggle notebook environment
Datasets	RFMiD (classification); DRIVE (vessel-prior pretraining)
Pretrained weights	DenseNet-121 (ImageNet); HRM-UNet (DRIVE); RETFound ViT-L/16 (trial)

CHAPTER 4
IMPLEMENTATION

4.1 TOOLS AND TECHNOLOGIES

4.1.1 Computing Environment

The complete pipeline is implemented in a single Jupyter notebook (version v8) executed on Kaggle. Training and evaluation ran on one NVIDIA Tesla T4 GPU with 15.6 GB of video memory, under PyTorch 2.11.0 with CUDA 12.8. The environment has no persistent storage between sessions, and session length is limited, which shaped several design choices (resumable caching and per-run result files, Section 4.2.6 and 4.2.10).

Table 4.1: Computing environment

Item	Specification
Platform	Kaggle notebook (Jupyter)
GPU	NVIDIA Tesla T4, 15.6 GB VRAM
Deep-learning framework	PyTorch 2.11.0 (CUDA 12.8 build), torchvision
Numerical precision	Automatic mixed precision (fp16 autocast), loss computed in fp32
Memory format	channels-last
Cache residency	2.14 GB of image and vessel arrays held in RAM

4.1.2 Datasets and Pretrained Models

Table 4.2: Datasets and pretrained components

Component	Source and use
RFMiD	3,200 colour fundus images with 46 labels; classification training, validation and test
DRIVE	Training data of the vessel segmenter only; not used directly in this notebook
HRM-UNet (hrm_unet_drive.pth)	Frozen vessel segmenter, 3.71 M parameters, 444 weight tensors, single-channel input and output

DenseNet-121 (ImageNet)	Classifier backbone, initialised from pretrained weights
RETFound ViT-L/16	Fundus-pretrained transformer used in the trial; weights are gated and require access approval

4.2 IMPLEMENTATION DETAILS

4.2.1 Data Discovery and Label Preparation

Archive layout differs between RFMiD mirrors, so image folders and label files are discovered rather than hard-coded. Candidate folders and CSV files are scored by whether their names contain the keywords of each split (train, valid/eval, test) and penalised for keywords of the other splits, and the best assignment is chosen. Each label row is mapped to an image path by file stem; rows without an image are dropped with a warning. Classes with fewer than 10 positives in the original training split are then removed, leaving 25 labels, and the training and validation sets are concatenated into a 2,560-image pool whose order is fixed so that cached arrays stay aligned.

Folds are produced by an implementation of iterative multilabel stratification: samples are visited rarest-label first and each is given to the fold that most needs its labels, with ties broken by the sample's other labels and then by fold size. The implementation asserts that every class appears in every fold and that the per-class positive count differs by at most two across folds. The realised spread was one positive, with five folds of 512 images.

4.2.2 Preprocessing

The function preprocess_pair decodes an image once, crops to the field of view using an intensity threshold of 10, and returns two aligned arrays: a CLAHE-enhanced 256×256 RGB array for the classifier and a plain grayscale 512×512 array for the segmenter. CLAHE (clip limit 2.0, 8×8 tiles) is applied to the L channel of the LAB image only. OpenCV CLAHE objects are not thread-safe, so each worker thread creates and keeps its own instance through thread-local storage; this is what allows the cache to be built with a thread pool.

4.2.3 Vessel Segmenter Integration

The HRM-UNet classes are copied unchanged from the segmenter's training notebook, because the saved weights are only valid for exactly that computation. Model dimensions are not hard-coded: the loader inspects the checkpoint's tensor shapes (for example, the first convolution (24, 1, 4, 4) and the last (1, 24, 1, 1)) to infer channel counts, and then loads the state dictionary strictly. Any missing or unexpected key raises an error; in the run reported here every key matched and the model has 3.71 M frozen parameters.

Inference is implemented as sliding-window prediction with torch unfold and fold: each 512×512 grayscale image is cut into 64×64 patches at stride 32, up to 512 patches are processed per forward pass, probabilities are obtained with a sigmoid, and overlapping predictions are averaged when the patches are folded back. The output is resized to 256×256 and stored as uint8. Before the full cache is built, a check on four images confirms that the output is plausible; the check measured a mean vessel probability of 0.270 and 17.6% of pixels above 0.5, and took 1.6 s for four images, which extrapolates to about 22 minutes for all 3,200 images.

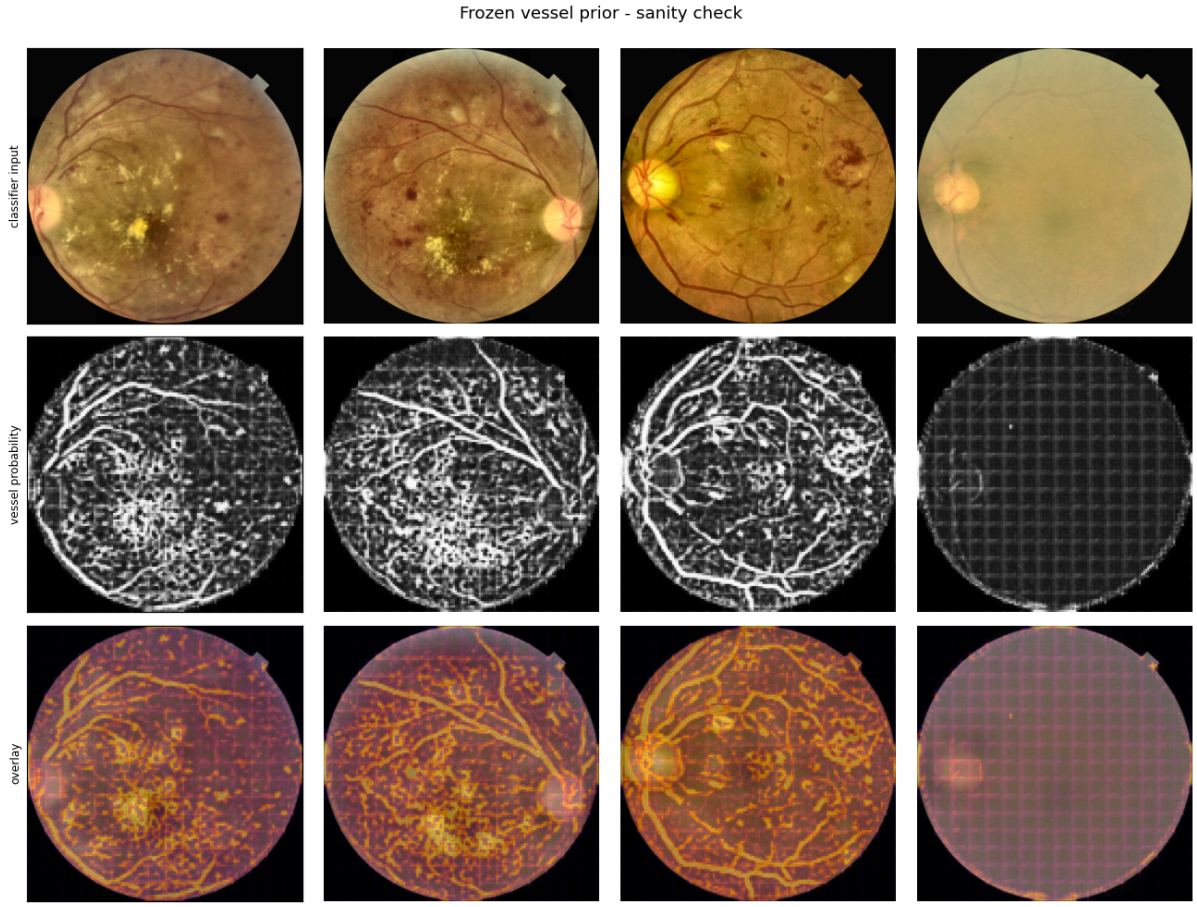


Figure 4.1: Example vessel maps from the frozen HRM-UNet

4.2.4 Caching

Because the segmenter is frozen, image preprocessing and vessel segmentation are done once and written to memory-mapped NumPy files: pool images (2560, 448, 448, 3) and vessel maps (2560, 256, 256), and the same for the 640 test images. The cache builder processes images in chunks of eight with a four-thread pool, and writes a small sidecar JSON file recording image count, cache sizes, vessel source and the number of rows completed. On restart, the builder checks the metadata and resumes from the recorded row using read-write memory-map mode, which preserves completed rows (write mode would truncate the file). A mismatch in any metadata field triggers a rebuild. An option allows previously computed vessel maps to be reused after their shape is validated against the dataset. After building, the arrays are loaded into RAM (2.14 GB), which

makes data loading effectively free.

4.2.5 Dataset Class and GPU Augmentation

The dataset class RFMiDCached is a view onto the cached arrays for a given set of indices (a fold or the full split). It returns the position inside the subset, not the global index, so per-sample state used by the curriculum remains aligned for any fold. Augmentation is executed on the GPU: image and vessel map are brought to a canvas of input size × 256/224 (area interpolation for the image, bilinear for the vessel map), stacked into one four-channel tensor, and transformed together by a single affine_grid and grid_sample call using random rotation of ±15°, scale of ±15%, shift of up to 5% and horizontal flip with probability 0.5. Photometric augmentation (channel gain ±6%, saturation ±15%, blur on 20% of samples, noise up to $\sigma = 0.02$) is applied to the RGB image only. Evaluation uses an identity transform and ImageNet mean and standard deviation normalisation.

4.2.6 Model Implementation

The classifier is implemented as VGDenseNet, which splits the torchvision DenseNet-121 feature extractor into stem, four dense blocks and three transition layers so that gates can be inserted after the transitions. Each VesselGate contains a 1×1 convolution to 16 hidden channels, batch normalisation, ReLU, and a second 1×1 convolution to the feature channel count; its output passes through a sigmoid and multiplies the features as $F \cdot (1 + \gamma A)$. The final convolution bias is zero-initialised and γ is a learnable scalar initialised to zero, so that the gated network equals the pretrained backbone at step 0. In evaluation mode each gate also stores its attention tensor, which is used by the interpretability tools. The head is global average pooling, dropout and a zero-bias linear layer.

The method param_groups builds the optimiser groups: backbone stages receive learning rates that decay by the layer-decay factor for each stage below the top, biases and normalisation parameters receive no weight decay, and the gates and head share a 10× higher rate. An assertion verifies that every trainable parameter belongs to exactly

one group. VGViT wraps a timm ViT-L/16, loads the RETFound weights (292 of 294 tensors matched; the two unmatched final-norm tensors are replaced by initialising the pooled-feature norm from the encoder norm), applies the same gate to the 14×14 token grid after blocks 8, 16 and 24, and uses gradient checkpointing to fit on a T4.

4.2.7 Loss Functions

Two loss classes share one interface, returning both a scalar for back-propagation and a detached per-sample loss for the curriculum. MultiLabelBCE wraps binary cross-entropy with logits, computed in fp32 (BCE on fp16 logits loses precision in the small-probability regime occupied by rare classes), with an optional per-class positive weight and optional per-sample weights. Positive weights are the capped (maximum 20) ratio of negatives to positives, computed for each fold from that fold's training labels only, and deferred re-weighting ramps them from one to their full value after half of the training epochs. AsymmetricLoss implements asymmetric focusing with separate exponents for positives and negatives and a probability margin that discards very easy negatives, and uses no positive weight.

4.2.8 Training Loop

The function train_recipe trains one recipe on one fold and returns held-out logits and test logits. For each fold the seed is offset by 101 times the fold number. Training uses AdamW, a cosine schedule with linear warm-up stepped every iteration, gradient clipping at 5.0, gradient scaling for mixed precision, and gradient accumulation when the batch is split (16 × 2 for inputs above 320 px). The best epoch is selected by validation mAP on the held-out fold, and its weights are restored at the end.

Optional components are switched by recipe fields. Mixup blends image and vessel pairs with a Beta-distributed coefficient and combines the losses against both label sets, instead of mixing the multi-hot labels, because ASL is defined for binary targets. Stochastic weight averaging uses torch's AveragedModel over the final quarter of training. Oversampling draws indices with replacement in proportion to class-rarity weights each epoch. ASL warm-up substitutes plain BCE for an initial fraction of

epochs. When the curriculum is on, the per-epoch sample weights are written by the scheduler and applied through the loss's sample-weight argument.4.2.11 Curriculum Scheduler

The CurriculumScheduler keeps, for each training sample, static class-difficulty signals (rarity of its rarest positive label, with median rarity for label-free images, and label cardinality) and dynamic sample-difficulty signals (an exponential moving average of its loss with coefficient 0.7, updated after every batch, and a vessel-misalignment value computed once per refresh). All signals are converted to percentile ranks by a rank-normalisation function, combined at two levels using the weights in Table 3.2, and mapped each epoch to a soft weight by a sigmoid around the pace threshold with a floor of 0.1. The product of this weight and the class-balance weight is the per-sample loss weight.

4.2.9 Calibration

Calibration is implemented as two small modules fitted by L-BFGS on binary cross-entropy. TemperatureScaler learns log T so that T stays positive, and VectorScaler learns a per-class scale (as log a) and offset b. The function fit_chain fits the temperature first, applies it, and then fits the class-wise scaler on the rescaled logits. A guard resets any non-finite parameter to the identity transform, so that a degenerate class with near-constant logits cannot introduce NaN values into the result tables. Cross-fitting is implemented by crossfit_probs: for each fold the calibrators are fitted on the logits of the other four folds and applied to the held-out fold, so no image is calibrated by a model that saw it. For the test split, the calibrators and the per-class F1 thresholds are fitted on all pooled OOF logits and applied to each fold model and to the ensemble.4.2.13 Metrics and Statistical Testing

Ranking metrics are computed with scikit-learn on a per-class basis and averaged over classes that have at least one positive. Head, mid and tail groups are defined from original-training prevalence at 5% and 1%. Expected calibration error uses equal-mass bins by default (15 bins), because equal-width bins place almost all predictions in the first bin. Per-class F1 thresholds are chosen on a grid of 49 values between 0.02 and 0.98

using out-of-fold predictions. Recipe comparisons use a paired bootstrap that resamples out-of-fold images (300 resamples), recomputes the mAP of both recipes on the same resample and reports the mean difference with a 95% interval. Test-split intervals for the best recipe are obtained by bootstrapping the 640 test images.

4.2.10 Interpretability and Uncertainty Tools

Grad-CAM is implemented with forward and backward hooks on the final normalisation layer of the DenseNet. It computes the gradient-weighted sum of feature maps for a chosen class, applies ReLU, upsamples to the input size and min-max normalises. A helper computes the Pearson correlation between each map and the vessel prior. Grad-CAM runs in fp32 with autocast disabled for stable gradients. Monte-Carlo dropout keeps the head's dropout layer in training mode while the rest of the network, including batch normalisation, stays in evaluation mode, and averages 20 stochastic passes to obtain a mean probability and a standard deviation per class.4.2.15 Reproducibility and Outputs

Seeds are set for Python, NumPy and PyTorch at the start and again for each fold. Every recipe and fold result is written to disk, and the final cells export summary tables (all recipes, per-class out-of-fold results, per-class sensitivity and specificity, literature comparison metrics), the fold assignment, a summary JSON with the full configuration, and the main figures. The output folder is then compressed into one archive.

CHAPTER 5
RESULTS AND DISCUSSION

This chapter reports the outcome of every experiment defined in Chapter 3, exactly as the notebook produced it. Results are given first, with the statistical evidence that supports each comparison, and are then interpreted in the discussion. Where an experiment did not support the hypothesis it was designed to test, or where the implementation itself was found to be flawed, this is reported as a finding and not omitted. Findings that qualify the headline claims are marked with a shaded callout.

5.1 EXPERIMENTAL SETUP

5.1.1 Hardware and Software

All experiments were run in a Kaggle notebook on a single NVIDIA T4 GPU (15 GB) using PyTorch, with automatic mixed precision and channels-last memory format. The frozen HRM-UNet vessel maps and the 448-pixel image cache were computed once and reused by every run. Per-fold training cost is given in Table 5.1.

Table 5.1: Training cost per fold on a single T4 GPU

Recipe	Epochs	Approx. time per fold	Approx. time per epoch
A_baseline (DenseNet-121, no vessel)	20	3.9 min	11 s
B_vessel (DenseNet-121 + vessel gates)	20	4.8 min	14 s
A_v7_regime (older, longer schedule)	30	5.5 min	11 s
A_retfound / B_retfound (ViT-L/16)	12	18.9 min	91–92 s

Vessel gating adds roughly 25–30% to epoch time. A ViT-L/16 epoch is about eight times slower than a DenseNet-121 epoch.

5.1.2 Data and Evaluation Protocol

The data, label filtering and cross-validation protocol are those described in Section 3.3. In brief: 25 RFMiD classes (8 head, 9 mid, 8 tail) are evaluated; the 2,560 training and

validation images form a development pool split into five multilabel-stratified folds of 512 images; the 640-image official test split is used only for final scoring. Two families of numbers are reported throughout:

- **OOF (out-of-fold) metrics,** computed on all 2,560 pool images, each predicted by a model that never trained on it. These are the primary basis for comparing recipes, because every tail class has a median of 20 pooled positives.
- **Test metrics,** computed on the official 640-image test split, either as the mean over the five fold models or for their five-model ensemble (suffix ens). Tail classes have only 3–7 test positives each, so test tail metrics are very noisy.

Recipe differences are judged with a paired bootstrap on OOF mAP (300 resamples). A difference is called better or worse only when its 95% confidence interval excludes zero.

5.1.3 Recipes Evaluated

Table 5.2: Recipes and the question each addresses

Recipe	Configuration	Folds	Purpose
A_v7_regime	DenseNet-121, BCE, 30 epochs, light regularisation (older regime)	5	Measure what the new regularisation regime bought
A_baseline	DenseNet-121, BCE, 20 epochs, new regime (layer-wise LR decay, weight decay 0.05, more dropout, stronger augmentation)	5	Reference for all comparisons
B_vessel	A_baseline + three vessel gates	5	Effect of vessel guidance
C_vessel_curric	B_vessel + two-level soft curriculum	5	Effect of curriculum
B_vessel_320px / 384px	B_vessel at higher input resolution	5	Effect of resolution
A_retfound / B_retfound	RETFound ViT-L/16 without / with vessel gates	1	Effect of a fundus-pretrained backbone
E0 … E5	Vessel + ASL + mixup + SWA, with one extra change each	3	Isolate which training changes help or hurt

5.2 RESULTS

5.2.1 Training Behaviour

Validation mAP rises steeply during the first five epochs and then flattens, with epoch-to-epoch noise of roughly ±0.01–0.02. Under the new regime the final training loss settles at about 0.16 (BCE), whereas under the older regime it falls to about 0.03. The best epoch varies widely between folds (for A_v7_regime, from epoch 15 to epoch 27), which is the reason early stopping was left disabled.

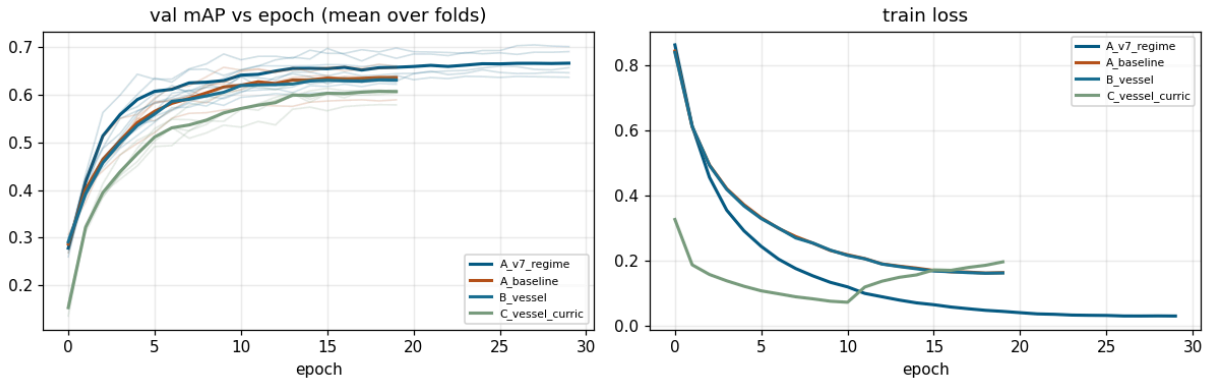


Figure 5.1: Validation mAP and training-loss curves for the Stage 1 recipes

5.2.2 Main Comparison (Stage 1, five-fold)

Table 5.3: Stage 1 results, out-of-fold (n = 2,560) and test-split ensemble (n = 640)

Recipe	OOF mAP	Head	Mid	Tail	OOF AUC	Test mAP (ens)	Test AUC (ens)	Test macro-F1 (ens)
A_v7_regime	0.6340	0.7926	0.6449	0.4631	0.9416	0.6217	0.9541	0.5197
A_baseline	0.6069	0.7731	0.6189	0.4271	0.9408	0.5770	0.9496	0.5185
B_vessel	0.5996	0.7797	0.6056	0.4127	0.9423	0.5766	0.9485	0.5153
C_vessel_curric	0.5757	0.7397	0.5767	0.4108	0.9178	0.5581	0.9221	0.4942

Fold-to-fold standard deviation of OOF mAP is 0.025, 0.026, 0.018 and 0.019 respectively. Per-fold test mAP (mean ± SD) is 0.574 ± 0.019, 0.553 ± 0.007, 0.552 ± 0.012 and 0.527 ± 0.003.

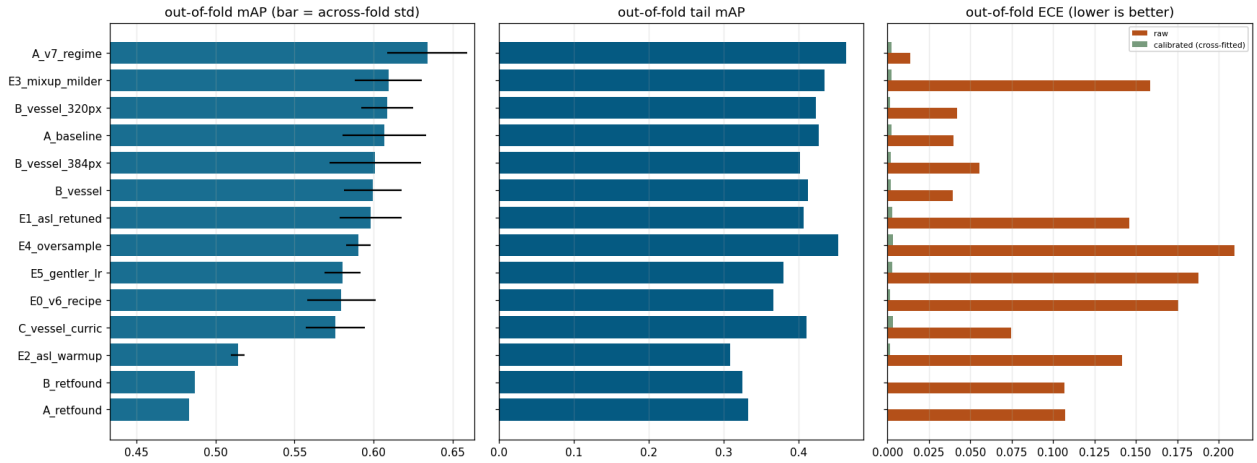


Figure 5.2: Overview comparison of recipes

Table 5.4: Paired bootstrap on OOF mAP, each recipe against A_baseline

Recipe	Δ mAP	95% CI	Verdict	Δ tail mAP	95% CI (tail)
A_v7_regime	+0.0271	[+0.0106, +0.0453]	Better	+0.0359	[−0.0088, +0.0820]
B_vessel	−0.0073	[−0.0201, +0.0055]	No clear difference	−0.0144	[−0.0527, +0.0241]
C_vessel_curric	−0.0311	[−0.0477, −0.0148]	Worse	−0.0164	[−0.0558, +0.0305]

5.2.3 Effect of the Regularisation Regime

The new regime (20 epochs, layer-wise learning-rate decay of 0.8, weight decay 0.05, head dropout 0.3, dense-layer dropout 0.1 and stronger photometric augmentation) was introduced to narrow the gap between training and test performance. Measured against the older regime on identical folds, it lowered OOF mAP by 0.027 (95% CI [−0.045, −0.011]). The older regime was better than the baseline on 19 of 25 classes, including 5 of 8 tail classes. Training loss under the new regime remains high (about 0.16, against about 0.03 under the older regime).

5.2.4 Effect of Vessel Guidance

Under the new regime, adding the three vessel gates to the baseline changed OOF mAP by −0.007 (95% CI [−0.020, +0.006]), which is indistinguishable from zero.

Test-ensemble mAP is essentially identical (0.5766 versus 0.5770). At the per-class level, vessel guidance improved out-of-fold average precision in 11 of 25 classes and in 2 of 8 tail classes relative to the baseline, and reduced it in the remainder; the largest gains were in PT (+0.079), ARMD (+0.026) and ODP (+0.022) and the largest losses in CRS (−0.061), MS (−0.060) and AH (−0.043). On the test split, diabetic-retinopathy sensitivity was higher with vessel guidance (0.831 versus 0.742) at a lower specificity (0.954 versus 0.971), while macro-F1 and tail-class F1 were slightly lower (0.515 versus 0.519 and 0.337 versus 0.366).

The earlier expectation that vessel guidance gives a consistent modest gain is therefore not reproduced under cross-validation. The fold-level spread of OOF mAP (SD about 0.02) is larger than the mean difference between A_baseline and B_vessel.

5.2.5 Effect of Curriculum Learning

Adding the two-level soft curriculum to B_vessel (recipe C) lowered OOF mAP by 0.031 relative to A_baseline (95% CI [−0.048, −0.015]), the clearest negative result among the Stage 1 comparisons. It also lowered OOF AUC from 0.941 to 0.918, test-ensemble AUC from 0.950 to 0.922, and increased raw (uncalibrated) OOF ECE from 0.039 to 0.075. It improved only 4 of 25 classes and 2 of 8 tail classes relative to the baseline. Its tail-class OOF mAP (0.411) is close to that of the baseline (0.427), and its test-ensemble tail mAP restricted to classes with at least five positives (0.513) is higher than the baseline's (0.494), although that subset has few positives and no confidence interval was computed.

5.2.6 Effect of Input Resolution

Table 5.5: Input-resolution sweep for B_vessel (five-fold)

Input size	OOF mAP	Tail mAP	Test mAP (ens)	Test macro-F1 (ens)	Test tail-F1 (ens)	Δ OOF mAP vs 224 px (95% CI)
224 px	0.5996	0.4127	0.5766	0.5153	0.3366	reference

320 px	0.6085	0.4232	0.5936	0.5051	0.2701	+0.0089 [−0.0081, +0.0230]
384 px	0.6010	0.4021	0.6045	0.4853	0.2206	+0.0014 [−0.0168, +0.0217]

Higher resolution gave a small increase in test-ensemble mAP and AUC (AUC 0.9485 to 0.9547) but no detectable OOF difference. The thresholded metrics moved in the opposite direction: macro-F1 and tail-class F1 both fell as resolution increased (tail-F1 from 0.337 to 0.221). Memory limits required splitting the batch at 384 px (16 × 2 accumulation), which is the only change besides resolution.

5.2.7 RETFound Trial

Table 5.6: RETFound trial on fold 0 (512 held-out images)

Recipe	Val mAP	Head	Mid	Tail	Val AUC	Test mAP	Raw OOF ECE
A_baseline (DenseNet-121)	0.6409	0.7770	0.6436	0.5016	0.9330	0.5628	0.0373
B_vessel (DenseNet-121)	0.6239	0.7758	0.6142	0.4829	0.9398	0.5693	0.0347
A_retfound (ViT-L/16)	0.4833	0.6769	0.4455	0.3321	0.9078	0.4409	0.1075
B_retfound (ViT-L/16 + gates)	0.4869	0.6905	0.4497	0.3250	0.9115	0.4412	0.1067

Against B_vessel on the same fold, A_retfound and B_retfound scored 0.141 and 0.137 mAP lower (bootstrap 95% CIs [−0.188, −0.082] and [−0.191, −0.082]). Validation mAP was still rising slowly when the 12-epoch budget ended (0.4823 at epoch 10, 0.4833 at epoch 11). Vessel gates on RETFound made no difference (0.4833 versus 0.4869). The checkpoint loaded 292 of 294 tensors; the two unmatched final-norm tensors were handled by initialising the pooled-feature norm from the encoder norm.

5.2.8 Ablation of Individual Training Changes

Table 5.7: One-at-a-time ablation on a vessel + ASL + mixup + SWA base (folds 0–2)

Recipe	Change relative to E0	OOF mAP	Tail mAP	Test mAP (ens)	Δ mAP vs E0 (95% CI)	Verdict
E0_v6_recipe	Base	0.5796	0.3660	0.5530	reference	-
E1_asl_retuned	ASL γ^- 4→3, clip 0.05→0.08	0.5982	0.4060	0.5685	+0.0186 [+0.0006, +0.0288]	Better
E2_asl_warmup	15% BCE warm-up	0.5142	0.3089	0.4727	−0.0654 [−0.0965, −0.0413]	Worse
E3_mixup_milder	mixup α 0.2→0.15, p 0.5→0.4	0.6093	0.4341	0.5755	+0.0297 [+0.0056, +0.0430]	Better
E4_oversample	Rarity-weighted resampling	0.5905	0.4522	0.5628	+0.0109 [−0.0200, +0.0342]	No clear difference
E5_gentler_lr	Lower LR, 3 warm-up epochs	0.5802	0.3795	0.5492	+0.0006 [−0.0169, +0.0143]	No clear difference

Two changes helped (milder mixup and retuned ASL), one clearly hurt (ASL warm-up), and two had no detectable overall effect. Rarity-weighted oversampling gave the largest tail gain (+0.086 tail mAP, CI [+0.0002, +0.1513], at the edge of significance) with no overall gain, and the lowest test-ensemble AUC of the group (0.912), consistent with memorisation of the few tail images. Compared directly with plain vessel-guided BCE (B_vessel) on the shared folds, no member of the ASL + mixup + SWA family was better: E0, E2 and E5 were worse on overall mAP, and E1, E3 and E4 were not distinguishable. Raw calibration was also far worse for this family (raw ECE 0.14–0.21 versus 0.04 for B_vessel), which is expected because ASL and mixup distort output probabilities.

5.2.9 Calibration

Table 5.8: Expected calibration error (equal-mass bins) before and after post-hoc calibration

Recipe	OOF raw	OOF calibrated (cross-fitted)	Test raw	Test calibrated (ens)
A_v7_regime	0.0138	0.0026	0.0124	0.0057
A_baseline	0.0398	0.0024	0.0355	0.0042
B_vessel	0.0393	0.0021	0.0349	0.0047
C_vessel_curric	0.0746	0.0033	0.0750	0.0032
B_vessel_320px	0.0422	0.0017	0.0379	0.0034
B_vessel_384px	0.0554	0.0020	0.0513	0.0045

Post-hoc calibration, fitted on out-of-fold predictions and applied cross-fitted, reduced pooled ECE by roughly an order of magnitude for every recipe: from 0.035–0.075 to below 0.006. For the best-test recipe (B_vessel_384px) the five-model ensemble reaches an ECE of 0.0045 and a Brier score of 0.0199 on the test split. Calibration is rank-preserving within each class, so it leaves mAP and AUC unchanged. The recipe with the lowest raw ECE (A_v7_regime, 0.014) was also the most accurate, while the curriculum recipe was the worst calibrated before correction (0.075).

5.2.10 Detailed Test-Split Performance

Table 5.9 reports the full metric set for the main recipes on the 640-image test split using the five-model ensemble, with out-of-fold calibration and thresholds. B_vessel_384px had the highest test mAP in this table and is used for the per-class analysis that follows.

Table 5.9: Test-split metrics, five-model ensemble

Metric	A_baseline	B_vessel	C_vessel_curric	B_vessel_320px	B_vessel_384px
mAP	0.5827	0.5756	0.5619	0.5965	0.6029
Macro AUC	0.9496	0.9483	0.9219	0.9540	0.9547
Macro-F1	0.5185	0.5153	0.4942	0.5051	0.4853
Micro-F1	0.7734	0.7878	0.7762	0.7972	0.7842
Macro sensitivity	0.5002	0.4920	0.4748	0.4760	0.4714

Macro specificity	0.9773	0.9805	0.9747	0.9781	0.9789
Per-label accuracy	0.9674	0.9700	0.9678	0.9709	0.9690
Exact-match accuracy	0.4938	0.5156	0.4484	0.5234	0.5094
Brier score	0.0210	0.0211	0.0224	0.0202	0.0199
ECE (calibrated)	0.0042	0.0047	0.0032	0.0034	0.0045
Screening AUC (normal vs abnormal)	0.9764	0.9753	0.9587	0.9754	0.9768
Screening sensitivity / specificity	0.947 / 0.843	0.939 / 0.858	0.941 / 0.754	0.964 / 0.806	0.935 / 0.858
DR AUC / F1	0.969 / 0.797	0.969 / 0.821	0.946 / 0.785	0.978 / 0.838	0.982 / 0.835

For B_vessel_384px, 95% bootstrap CIs over the 640 test images are: macro AUC 0.955 [0.940, 0.967], micro-F1 0.784 [0.765, 0.802], macro-F1 0.485 [0.427, 0.529]. The mAP in this table (0.6029) differs slightly from the ensemble mAP in Table 5.5 (0.6045) because the two values come from separate evaluation cells; the discrepancy was not investigated.

Per-label accuracy of 0.97 is inflated by the large number of true negatives and should not be read as quality; macro-F1 and macro sensitivity (both below 0.52) describe the model's behaviour on the diseases far better. Table 5.10 gives the per-class results of B_vessel_384px.

Table 5.10: Per-class test results, B_vessel_384px (threshold chosen on OOF predictions)

Class	Group	Test positives	Sensitivity	Precision	F1	AUC
Disease_Risk	Head	506	0.935	0.961	0.948	0.977
DR	Head	124	0.798	0.876	0.835	0.982
MH	Head	104	0.875	0.771	0.820	0.969

ODC	Head	91	0.692	0.583	0.633	0.909
TSLN	Head	53	0.717	0.567	0.633	0.967
DN	Head	46	0.326	0.405	0.361	0.888
MYA	Head	32	0.750	0.889	0.814	0.995
ARMD	Head	31	0.710	0.611	0.657	0.972
ODP	Mid	24	0.458	0.423	0.440	0.948
BRVO	Mid	23	0.739	0.630	0.680	0.947
ODE	Mid	17	0.706	0.923	0.800	0.975
LS	Mid	15	0.467	1.000	0.636	0.937
RS	Mid	14	0.643	0.600	0.621	0.988
CSR	Mid	13	0.308	0.667	0.421	0.977
CRS	Mid	11	0.273	0.500	0.353	0.949
CRVO	Mid	9	0.556	1.000	0.714	0.987
RPEC	Mid	4	0.000	0.000	0.000	0.925
MS	Tail	7	0.000	0.000	0.000	0.941
ERM	Tail	5	0.600	0.158	0.250	0.922
AH	Tail	5	0.400	0.667	0.500	0.994
RT	Tail	5	0.000	0.000	0.000	1.000
AION	Tail	4	0.250	0.500	0.333	0.937
PT	Tail	4	0.000	0.000	0.000	0.975
EDN	Tail	4	0.250	0.143	0.182	0.946
MHL	Tail	3	0.333	1.000	0.500	0.863

Performance is strong on the large head classes (DR, MH, Disease_Risk: F1 0.82–0.95) and falls steeply with class size. Four classes have zero sensitivity and zero F1 (RPEC, MS, RT and PT), and three further tail classes have F1 at or below 0.25 (ERM, AION, EDN). Yet the ranking quality for these same classes is high (AUC 0.92–1.00, including a perfect 1.000 for RT, which has five positives). The model therefore ranks rare-disease images well but the thresholds, chosen on out-of-fold predictions, do not convert that ranking into correct decisions on the test split. The head class DN (46 test positives) is a notable weak point, with sensitivity 0.33 and AUC 0.89.

5.2.11 Interpretability and Vessel Agreement

Grad-CAM maps were generated from the fold-0 B_vessel model for one vascular class (diabetic retinopathy) and one non-vascular class (macular hole). The mean correlation between the class activation map and the frozen vessel prior was 0.182 for DR and 0.286 for MH. The learned gate coefficients were negative at all three scales: $\gamma = -0.103$ (after transition 1), -0.222 (transition 2) and -0.270 (transition 3).

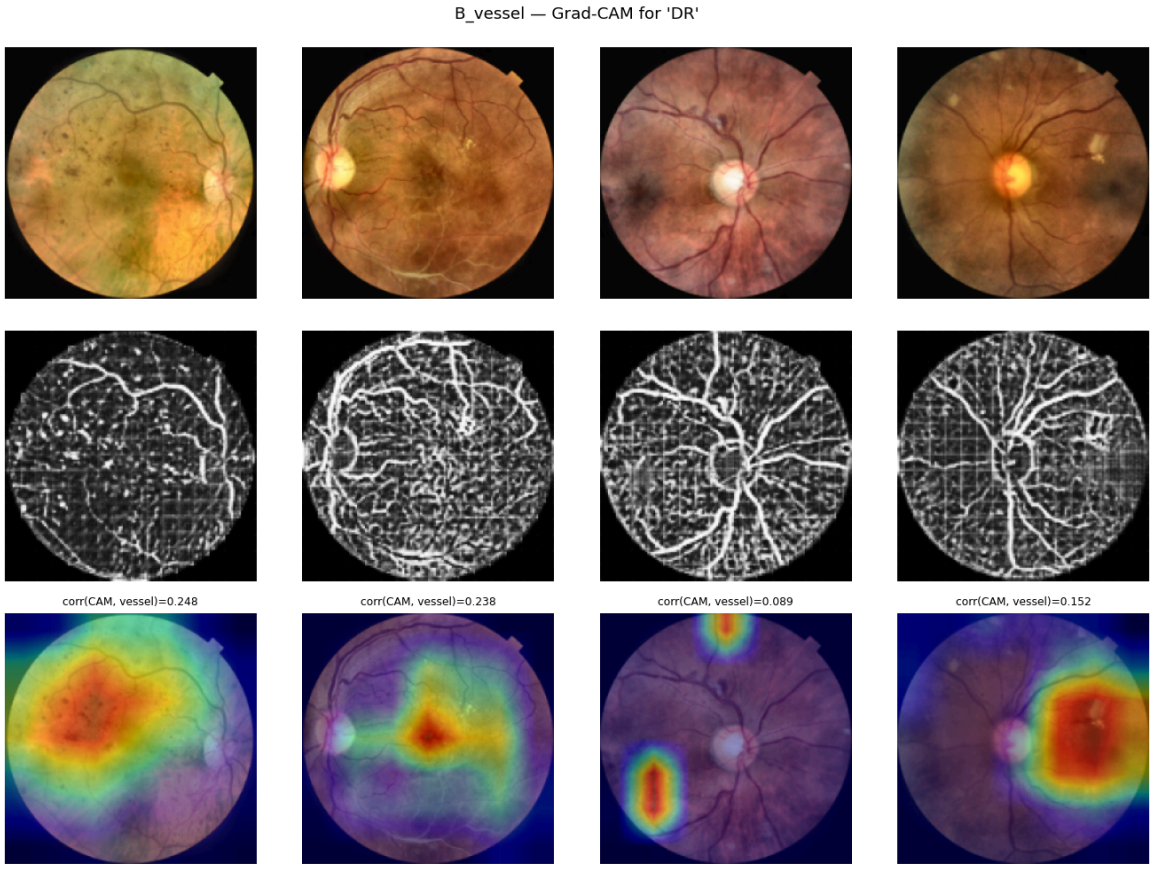


Figure 5.3: Grad-CAM overlays for diabetic retinopathy (vascular class)

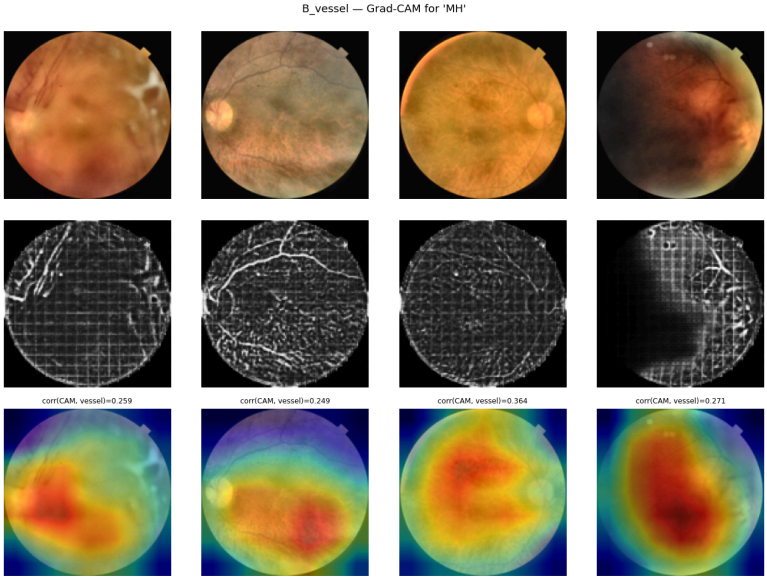


Figure 5.4: Grad-CAM overlays for macular hole (non-vascular class)

5.2.12 Uncertainty Triage

Monte-Carlo dropout (20 passes) was applied to the fold-0 B_vessel model on the test split, and images were ranked by mean predictive standard deviation. Deferring the most uncertain images to a human reader raised the mAP on the remaining images from 0.569 (no deferral) to 0.598 at 20% deferral and 0.693 at 50% deferral.

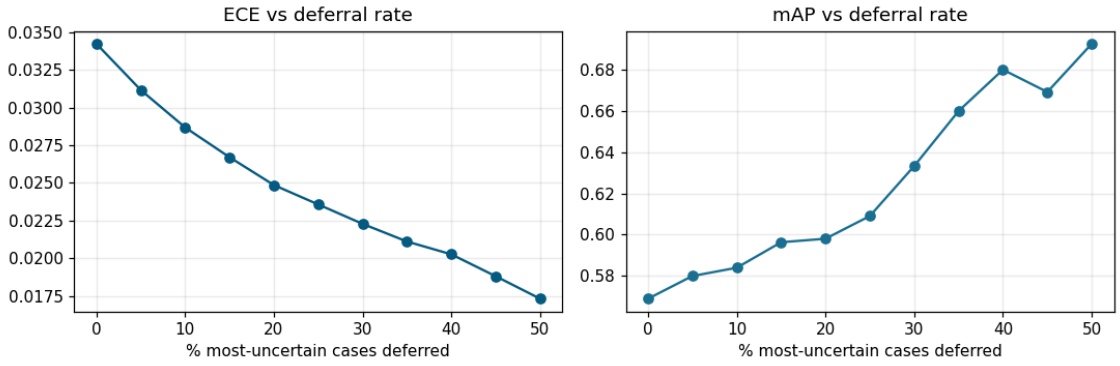


Figure 5.5: Uncertainty-based deferral curve

The monotone rise indicates that the MC-dropout spread carries information about which images are harder. The result is from a single model on one split, no random-deferral reference curve is included, and mAP over a shrinking subset of images

with few tail positives becomes noisier as the retained set shrinks; the gain at 50% deferral should be read with that in mind.

5.2.13 Comparison with Published Work

Published RFMiD studies report macro-F1 of 0.70–0.74 and accuracy of 75–88%, and one reports ECE of 0.065. The present macro-F1 (0.485 for B_vessel_384px, 0.520 for A_baseline) and ECE (0.0045) are not comparable like-for-like: the cited works use 8–10 classes and often single-label accuracy, whereas this work evaluates 25 multi-label classes including eight rare ones, with different calibration binning. The closest like-for-like quantity is the normal-versus-abnormal screening task, where this system reaches AUC 0.977, sensitivity 0.935 and specificity 0.858. Diabetic-retinopathy AUC (0.982) is comparable in magnitude to published APTOS-2019 results, but that is a different dataset.

Table 5.11: Comparison with published work

Study	Dataset and task	Acc.	Macro-F1	AUC	Sens. / Recall	ECE
Perumalsamy et al. (2026)	RFMiD, 8-class	0.754	0.743	–	–	0.065
Sekhar et al. (2026)	APTOS, binary DR	0.891	–	–	0.989	–
Sekhar et al. (2026)	APTOS, 5-grade DR	0.823	about 0.60	–	–	–
Venkatappa et al. (2026)	RFMiD, 10-class	0.885	0.697	–	–	–
Anusha et al. (2026)	RFMiD, 37-label multi-label	0.97	–	–	–	–
Bano et al. (2025)	LVPEI, 5-grade DR	0.900	–	–	0.890	–

Hybrid ResNet-UNet	APTOS, DR	0.852	–	0.946	–	–
This work	RFMiD, 25-label multi-label	–	0.485	0.955	0.935	0.002 to 0.006

5.3 DISCUSSION

5.3.1 Summary of Findings

Table 5.12: Outcome of each hypothesis

Hypothesis	Evidence	Outcome
The new regularisation regime narrows the generalisation gap	ΔmAP −0.027 [−0.045, −0.011] versus the older regime	Not supported; accuracy fell
Vessel guidance improves classification	ΔmAP −0.007 [−0.020, +0.006]; 11/25 classes improved	Not supported (no detectable effect)
Vessel guidance is used more on vascular classes	CAM–vessel correlation DR 0.18 < MH 0.29; $\gamma < 0$ at all scales	Not supported
Curriculum learning improves tail performance	ΔmAP −0.031 [−0.048, −0.015]; vessel-confidence signal degenerate	Not supported; implementation flaw found
Higher resolution improves small-lesion classes	ΔmAP +0.009 and +0.001, CIs span zero	Not supported (inconclusive)
A fundus-pretrained ViT-L beats DenseNet-121	−0.14 mAP on fold 0	Not supported under tested settings
Post-hoc calibration improves probability quality	Pooled ECE 0.035–0.075 → below 0.006	Supported
MC-dropout uncertainty is useful for triage	mAP 0.569 → 0.693 at 50% deferral (one model)	Supported (preliminary)
Individual training changes can be separated	Milder mixup and retuned ASL helped; ASL warm-up hurt	Partly supported (3 folds; interactions untested)

5.3.2 Interpretation

The simplest configuration, a DenseNet-121 trained for 30 epochs with binary cross-entropy, matched or beat every more elaborate variant, which makes it a strong, reproducible baseline. Three observations help explain why the added components did not pay off.

1. **Data scale.** With about 2,000 training images per fold and 20 pooled positives per tail class, an added component must exceed the fold-to-fold spread (SD about 0.02 mAP). Effects below 0.03 sit at the limit of detection even with 2,560 out-of-fold predictions. Cross-validation and the paired bootstrap were therefore essential to avoid reporting noise as progress.
2. **Regularisation.** The new regime's training loss stayed near 0.16, against 0.03 for the older regime, and validation mAP plateaued lower. Combining five regularisers with a shorter schedule plausibly limited fitting more than it limited overfitting. This was not tested directly.
3. **Fusion design.** The gate attention is computed only from the vessel map, so it adds a learned spatial weighting but no image-dependent information. The network chose negative γ values, so with this fusion the vessel prior acted as a nuisance signal on average. A fairer test would fuse vessel information in an image-dependent way, or apply it only to vascular classes.

The strongest positive result is calibration. A simple post-hoc scaling fitted on out-of-fold predictions removes most miscalibration, even when the raw probabilities were badly distorted (raw ECE 0.14–0.21 for the ASL and mixup recipes). Calibration can therefore be treated separately from the choice of training loss.

5.3.3 Limitations and Scope

1. **Vessel guidance and curriculum learning:** The best recipe (A_v7_regime) uses neither. The curriculum signal is nearly constant (mean −0.996, SD 0.002), making its difficulty term ineffective. A per-sample Grad-CAM–vessel correlation is a potential improvement.
2. **Confounded attribution:** Five regularisation changes were bundled, preventing

identification of the cause of accuracy loss. Vessel guidance was not tested under the better 30-epoch regime.

3. **Rare-class thresholds:** Four of 25 classes have zero test sensitivity, while three others have F1 ≤ 0.25. Out-of-fold thresholds do not transfer reliably to rare classes.
4. **Limited tail samples:** Tail-class metrics rely on only 3–7 positives, and the macro-F1 95% CI is wide (0.427–0.529).
5. **Label filtering:** Restricting RFMiD to 25 classes excludes 21 rare diseases, limiting conclusions and comparisons with studies using different label sets.

5.3.4 Threats to Validity

- **Internal:** A single seed per recipe limits confidence in differences below approximately 0.02 mAP. Untuned hyperparameters may disadvantage some methods, including RETFound.
- **Construct:** Macro metrics are unstable for rare classes, and thresholded F1 depends on thresholds fitted on another split.
- **External:** Evaluation uses only RFMiD and one vessel segmenter, without external validation across cameras or populations.

5.3.5 Implications

The strongest model is a plain DenseNet-121 using the older training schedule and post-hoc calibration, achieving test-ensemble mAP of 0.62 and macro AUC of 0.954. Calibration and uncertainty-based deferral show practical value, whereas vessel guidance and curriculum learning remain unvalidated. Future work should improve vessel fusion and difficulty estimation, test the revised approach under the better training regime, and use multiple random seeds.

Chapter 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

This work investigated whether a frozen vessel prior, a difficulty-based training curriculum and post-hoc uncertainty calibration can improve multi-label retinal disease classification on the long-tailed RFMiD dataset. Of the 46 labels supplied, 25 had at least ten training positives and were retained (8 head, 9 mid and 8 tail classes; imbalance ratio 138:1). A frozen HRM-UNet (3.71 M parameters, trained on DRIVE) produced vessel probability maps that were fused into a DenseNet-121 backbone through three gamma-gated residual attention modules. Calibration used a chained temperature and class-wise scaling stage, followed by MC-dropout for uncertainty estimation.

A central contribution is methodological. The earlier single-split protocol had too few tail positives (a median of 4 in the test split) to separate real effects from noise. The 2,560 development images were therefore evaluated with five-fold multilabel-stratified cross-validation. Calibration and thresholds were fitted on out-of-fold predictions, and paired bootstrap confidence intervals were used to compare recipes. The 640-image official test split was used only for final scoring. This raised the median number of tail positives to 20 and allowed each claim to be stated with its uncertainty. The principal findings are summarised in Table 6.1.

Table 6.1: Paired bootstrap comparisons on out-of-fold mAP

Component	**Comparison (5-fold CV)**	**Change in OOF mAP (95% CI)**	**Verdict**
Vessel guidance	B_vessel vs A_baseline	-0.007 [-0.020, +0.006]	No clear difference
Curriculum learning	C_vessel_curric vs A_baseline	-0.031 [-0.048, -0.015]	Improved Rare classes but overall worse
Training regime	A_v7_regime (30 ep.) vs A_baseline (20 ep.)	+0.027 [+0.011, +0.045]	Better
Input resolution	320 px vs 224 px	+0.009 [-0.008, +0.023]	No clear difference

Input resolution	384 px vs 224 px	+0.001 [-0.017, +0.022]	No clear difference
Milder mixup (3 folds)	E3 vs E0	+0.030 [+0.006, +0.043]	Better
ASL re-tuning (3 folds)	E1 vs E0	+0.019 [+0.001, +0.029]	Better
ASL warm-up (3 folds)	E2 vs E0	-0.065 [-0.097, -0.041]	Worse

6.1.1 Key findings

- **Calibration was the most dependable gain.** Post-hoc calibration cut expected calibration error (ECE) from 0.035–0.075 (binary cross-entropy models) and 0.14–0.21 (Asymmetric Loss models) to 0.002–0.006 for every five-fold DenseNet recipe, with ranking metrics unchanged. Because the reported ECE pools many easy negatives, it should be read alongside per-group ECE, and it is not directly comparable with values reported for smaller label sets.
- **Uncertainty estimates support clinical triage.** Deferring the most uncertain 20% and 50% of cases raised mAP on the remaining cases from 0.569 to 0.598 and 0.693, which shows MC-dropout uncertainty carries real information for a human-in-the-loop workflow.
- **The final model is strong on screening and common diseases.** At 384 px, diabetic retinopathy reached AUC 0.982 and F1 0.835. Normal-versus-abnormal screening reached sensitivity 0.935, specificity 0.858 and AUC 0.977. Macro AUC was 0.955 and macro-F1 was 0.485 (95% CI 0.427–0.529).
- **Rare classes are well ranked but need better thresholds.** Three tail classes (MS, PT and RT) had AUC of 0.94–1.00 but zero sensitivity at the chosen thresholds. The model separates these classes well, and the shortfall lies in F1 thresholds fitted from very few positives.
- **Vessel guidance showed a class-specific benefit but no overall gain.** It changed out-of-fold mAP by −0.007 (95% CI −0.020 to +0.006), so there was no clear overall effect. On the test split it raised diabetic retinopathy sensitivity from 0.74 to 0.83 (ensemble, 224 px). The learned gate strengths were negative at all three

scales, so the interpretability check did not support the original hypothesis. These results come from one fold-0 model and two classes, so they show a lack of support rather than a refutation.

- **The curriculum did not help, and the cause is identified.** It scored −0.031 mAP against the baseline (CI −0.048 to −0.015) and improved 4 of 25 classes. The soft pacing did fix the earlier exclusion of rare classes (all 25 classes received gradient from epoch 0, against 4 of 46 in the hard pool). The vessel-confidence signal was effectively constant (SD 0.002), so it ranked samples by noise. A more informative difficulty signal is the clear next step.
- **Simpler training choices held up well.** The original 30-epoch regime outperformed the heavier 20-epoch one (OOF mAP 0.634 against 0.607). The regimes differ in epochs as well as regularisation, so the cause cannot be isolated. Higher resolution (320 and 384 px) raised test-ensemble mAP from 0.577 to 0.594 and 0.605, but at about 2.5× the cost per fold and with differences within noise out-of-fold. A fundus-pretrained RETFound ViT-L/16 reached an OOF mAP of about 0.48 against 0.64 for DenseNet-121 in a single-fold, 12-epoch trial. Loss and augmentation changes gave only small differences, and none beat plain vessel-guided binary cross-entropy.

Overall, this project built and rigorously tested a vessel-guided, curriculum-based pipeline for long-tailed retinal classification on RFMiD. Controlled, fold-wise evaluation showed that calibration and uncertainty-based deferral are reliable gains, and that vessel guidance and curriculum learning need stronger signals before they can be called improvements. Reporting these outcomes openly gives a trustworthy basis for future work. Because the study filtered out 21 rare labels, its mAP values are best compared within this study, not directly with work that uses other label sets

6.1.2 Limitations

The study uses a single dataset and 25 of its 46 labels. Test-split tail classes contain only 3 to 7 positives, so per-class test metrics are very uncertain. The loss and augmentation ablation used three of five folds, and the RETFound trial one fold. Out-of-fold scores are mildly optimistic because the best epoch was chosen on the held-out fold, which is why

test-split numbers are reported alongside them. The vessel segmenter was trained on DRIVE only and applied to RFMiD without adaptation, and the interpretability analysis rests on one model and two classes.

6.2 Future Work

The findings suggest several directions, ordered roughly by how directly they follow from the results above.

- **Repair the curriculum.** Replace the constant vessel-confidence signal with a per-sample Grad-CAM-to-vessel correlation. Also run a curriculum-without-deferred-re-weighting ablation to test whether stacking rarity weights with re-weighting caused the degradation.
- **Test whether the vessel prior is used.** Add control runs with shuffled or blank vessel maps, and re-examine the gate design given the negative learned γ values. Richer priors (optic disc, macula, lesions) and a segmenter adapted beyond DRIVE are natural extensions.
- **Strengthen the statistics.** Repeat cross-validation with five to ten seeds, complete the loss and augmentation ablation on all folds, and run the 45-epoch and combined recipes to check for interaction effects.
- **Address the tail directly.** Recover the 21 excluded classes using few-shot learning, grouped labels or synthetic data. Select thresholds with confidence intervals or cost-sensitive criteria, not F1 maximisation on a few positives.
- **Validate beyond RFMiD.** Evaluate frozen on ODIR-5K and other multi-centre datasets after mapping their categories to the RFMiD label space, and examine robustness across cameras and populations.

REFERENCES

[1] Z. M. Al-johani, H. Ghandorh, and W. M. S. Yafooz, "Lightweight deep learning models for retinal disease classification: A survey," in Proc. 2026 4th Int. Conf. Artif. Intell. Mach. Learn. Appl. (AIMLA), 2026, pp. 1–8, doi: 10.1109/AIMLA67915.2026.11522510.

[2] F. Bano, K. Pranavika, A. S. Kiran, and M. D. Reddy, "Attention U-Net and CNN-based deep learning framework for diabetic retinopathy severity prediction and retinal vessel segmentation," in Proc. 2025 Int. Conf. Comput. Robot., Test. Eng. Eval. (ICCRTEE), 2025, pp. 1–6, doi: 10.1109/ICCRTEE64519.2025.11052950.

[3] J.-M. Hou, C.-K. Lee, Y.-A. Lin, and P.-H. Tseng, "A semi-supervised retinal vessel segmentation method via adaptive uncertainty estimation," in Proc. 46th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC), 2024, pp. 1–5, doi: 10.1109/EMBC53108.2024.10782670.

[4] Q. Huang and W. Yu, "HRM-UNet: Image segmentation of retinal blood vessels on the basis of improved VM-UNet," in Proc. 5th Int. Conf. Mach. Learn. Comput. Appl. (ICMLCA), 2024, pp. 286–289, doi: 10.1109/ICMLCA63499.2024.10754456.

[5] A. J. Kadhim, H. Seyedarabi, R. Afrouzian, and F. S. Hasan, "Diabetic retinopathy classification using hybrid color-based CLAHE and blood vessel in deep convolution neural network," IEEE Access, vol. 12, pp. 194750–194761, 2024, doi: 10.1109/ACCESS.2024.3519361.

[6] K. Meenakshi, Shakeeluddin, and A. Kumar, "A hybrid deep learning model integrating segmentation and feature extraction for diabetic retinopathy detection," in Proc. 2025 Int. Conf. In-tell. Secure Eng. Solutions (CISES), 2025, pp. 533–539, doi: 10.1109/CISES66934.2025.11265529.

[7] A. P K, N. Ajith, L. Z. J, and J. J, "Hybrid attention-driven and explainable AI for multi-label retinal disease diagnosis," in Proc. 2026 IEEE Int. Conf. Emerg. Comput. Intell. Technol. (ICoECIT), 2026, pp.1–6, doi: 10.1109/ICoECIT68303.2026.11497847.

[8] M. Perumalsamy, A. S. J, A. M. V, A. Prabhu, T. C, and T. P, “CLARE-Net: A validation-driven adaptive ensemble framework for reliable multi-class retinal disease classification,” in Proc. 2026 9th Int. Conf. Comput. Methodol. Commun. (ICCMC), 2026, pp. 2075–2080, doi: 10.1109/ICCMC69250.2026.11624802.

[9] S. Sadhukhan, S. Das, A. Tarafdar, and A. Singh, “Towards explainable retinal vessel segmentation: A deep learning approach,” in Proc. 2025 IEEE 6th India Council Int. Subsections Conf. (INDISCON), 2025, pp. 1–5, doi: 10.1109/INDISCON66021.2025.11254539.

[10] B. S. Sekhar, J. N. Kutikuppala, V. R. Pilli, and D. BalaKotaiah, “Deep learning-based detection and severity classification of diabetic retinopathy using fine-tuned DenseNet with Grad-CAM,” in Proc. 2026 5th Int. Conf. Comput. Netw., Big Data IoT (ICCBI), 2026, pp. 602–607, doi: 10.1109/ICCBI68589.2026.11619719.

[11] U. Venkatappa, S. Mukherjee, S. S. Dinde, S. Hathi, Sinchana, and S. Singh, “Fundus image analysis for age related retinal disease detection using deep learning and XAI methods,” in Proc. 2026 13th Int. Conf. Comput. Sustain. Global Dev. (INDIACom), 2026, pp. 1–6, doi: 10.23919/INDIACom70271.2026.11525785.

[12] C. Wang, M. Sun, and Y. Ma, “A new strategy for retinal vessel segmentation based on semi-dilation supervised learning and boundary reconstruction,” in Proc. 2024 7th Int. Conf. Pattern Recognit. Artif. Intell. (PRAI), 2024, pp. 775–780, doi: 10.1109/PRAI62207.2024.10826864.

[13] J. Wang, H. Peng, S. Chen, and S. Ren, “Ensemble learning for retinal disease recognition under limited resources,” Med. Biol. Eng. Comput.,2024, doi: 10.1007/s11517-024-03101-3.

[14] Y. Zhou and X. Xu, “Retinal blood vessel image segmentation method based on improved U-Net,” in Proc. 2025 8th Int. Conf. Artif. Intell. Big Data (ICAIBD), 2025, pp. 695–700, doi: 10.1109/ICAIBD64986.2025.11082026.

[15] Md. M. Rahman and J. Rahman, “An ensemble transfer learning framework for multi-class retinal fundus disease classification,” in Proc. 2026 Int. Conf. Power, Electron., Commun., Comput.,Intell. Infrastruct. (PECCII), Pabna,

Bangladesh, Jun. 2026, doi: 10.1109/PECCII70991.2026.11662044.

[16] Md. M. H. Rokon, Md. O. Faruk, M. K. Saha, Md. S. Hossain, S. S. M. Pranto, Md. M. Hasan, M. Mamun, and Md. R. Hasan, "Deep learning-based multiclass classification of eye diseases using retinal fundus images," in Proc. 2026 Int. Conf. Power, Electron., Commun., Comput., Intell. Infrastruct. (PECCII), Pabna, Bangladesh, Jun. 2026, doi: 10.1109/PECCII70991.2026.11661832.

[17] M. James, B. Ramamurthy, and Saravanan K. N., "DeepRetina: Transformer-enhanced EfficientNet for retinal disease classification," in Proc. 2026 6th Int. Conf. Adv. Electr., Comput., Commun. Sustain. Technol. (ICAECT), 2026, doi: 10.1109/ICAECT68478.2026.11426056.

[18] H. Acharya, S. Adhikari, and S. B. Thapa, "Retinal fundus disease detection and classification using ResNet-50," in Proc. 2025 Int. Conf. Inf. Sci. Commun. Technol. (ICISCT), 2025, pp. 74–79, doi: 10.1109/ICISCT68600.2025.11441375.

[19] Sandhya B. R. and N. Nalini, "Retinal vessel segmentation through green channel enhancement and comparative deep learning architectures," in Proc. 2026 4th Int. Conf. Knowl. Eng. Commun. Syst. (ICKECS), Apr. 2026, doi: 10.1109/ICKECS70176.2026.11527623.

[20] M. I. S. Watson Benjamin, K. B. Gutti, A. Maram, S. Ravikumar, N. Krishnaveni, and K. Maithili, "Vision transformer based multi-class eye disease classification using retinal fundus images," in Proc. 2026 5th Int. Conf. Sentiment Anal. Deep Learn. (ICSADL), 2026, pp. 548–553, doi: 10.1109/ICSADL67539.2026.11451800

Appendices

Appendix A: Experimental Environment and Reproducibility

All experiments were run in a single Jupyter notebook (version v8) on Kaggle. Table A.1 lists the environment and the settings that determine reproducibility. Every finished fold was cached on disk by recipe and fold, so interrupted runs resumed without repeating completed work.

Table A.1: Environment and reproducibility settings

Item	**Value**
Hardware	NVIDIA Tesla T4 GPU, 15.6 GB VRAM
Framework	PyTorch 2.11.0 (CUDA 12.8), mixed-precision (AMP) training with fp32 loss
Random seeds	Training seed 42; cross-validation split seed 42
Cross-validation	5-fold multilabel-stratified over 2,560 pooled images (512 per fold); class positives differ by at most 1 across folds
Bootstrap	300 paired resamples of out-of-fold images for recipe-versus-recipe comparison; 95% intervals
Image cache	Pool images 2560 x 448 x 448 x 3 (uint8), vessel maps 2560 x 256 x 256; test split 640 images; about 2.1 GB held in RAM
Vessel segmenter	HRM-UNet, 3.71 M parameters, frozen; 64 x 64 patches, stride 32, run at 512 px and downsampled to 256 px
Compute time (T4)	Stage 1 (4 recipes x 5 folds) 95.8 min; resolution sweep 112.4 min; RETFound trial 37.8 min; loss/augmentation ablation (3 folds) 91.4 min

Appendix B: Hyperparameter Configuration

Table B.1: Main hyperparameters (v8 regime)

Parameter	**Value**	**Note**
Backbone	DenseNet-121	ImageNet-pretrained; 6.99 M trainable parameters
Input size	224 px (320, 384 in sweep)	Images cached at 448 px
Vessel fusion	After transition 1, 2, 3	Gate: f x (1 + gamma x A), gamma initialised to 0
Epochs	20	Cosine schedule; 1 warm-up epoch
Batch size	32	Effective 32 for RETFound (16 x 2)

Optimiser	AdamW	Weight decay 0.05 (none on biases and norms)
Learning rates	1e-4 backbone; 1e-3 gates and head	Layer-wise decay 0.8 per stage
Dropout	Head 0.3; dense layers 0.1	Head dropout also used for MC-dropout
Gradient clipping	5.0	
Positive weight cap	20	Capped inverse-frequency, computed per fold
Deferred re-weighting	Starts at 50% of training	
Curriculum	Warm-up stage 25%; pace 0.60 to 1.0 by 75%	Soft sigmoid weighting, floor 0.1, temperature 0.12
Difficulty weights	Rarity 0.15; cardinality 0.15; loss 0.55; vessel misalignment 0.15	Loss EMA 0.7
CLAHE	Clip limit 2.0, grid 8	Classifier view only; segmenter uses plain grayscale
Augmentation	Rotation +/-15 deg, zoom, shift, h-flip, brightness/contrast/gamma, channel gain +/-6%, saturation +/-15%, blur, noise (sigma up to 0.02)	No cutout, no vertical flip
Calibration	Temperature then class-wise scaling	Fit on out-of-fold logits; 15 equal-mass bins
MC-dropout	20 samples	Head dropout only; BatchNorm in eval mode
F1 thresholds	49-point grid, 0.02 to 0.98	Per class, fit on out-of-fold predictions
ASL (recipe E)	gamma+ 1, gamma- 4, clip 0.05	Mixup alpha 0.2, probability 0.5; SWA over last 25%
RETFound trial	ViT-L/16, 12 epochs, fold 0	Layer decay 0.75, drop-path 0.2, gates after blocks 8, 16, 24

Table B.2: Recipe definitions

Recipe	**Definition**
A_baseline	DenseNet-121, binary cross-entropy, v8 regime (20 epochs)
A_v7_regime	As A_baseline, with the earlier v7 settings (30 epochs, weak weight decay, head dropout 0.2)
B_vessel	A_baseline plus frozen vessel prior and gated fusion at three scales
C_vessel_curric	B_vessel plus two-level soft curriculum
B_vessel_320px, _384px	B_vessel at 320 and 384 px input

A_retfound, B_retfound	RETFound ViT-L/16 without and with vessel gates (fold 0 only)
E0_v6_recipe	Vessel guidance with Asymmetric Loss, mixup and SWA
E1 to E5	E0 with one change each: E1 re-tuned ASL (gamma- 3, clip 0.08); E2 BCE warm-up for 15% of epochs; E3 milder mixup (alpha 0.15, p 0.4); E4 rarity-weighted oversampling; E5 gentler learning rates and 3 warm-up epochs

Appendix C: Dataset Details and Label Abbreviations

RFMiD contains 3,200 images: 1,920 for training, 640 for validation and 640 for testing, with 46 labels as supplied (Disease_Risk and 45 diseases). Labels with fewer than 10 positives in the original training split were removed, leaving 25 labels (Disease_Risk and 24 diseases). Training and validation images were pooled (2,560) for cross-validation; the test split was kept untouched. Head, mid and tail groups were defined from training prevalence (head 5% or more, tail 1% or less), giving 8 head, 9 mid and 8 tail labels, with an imbalance ratio of 138:1.

Appendix D: Code Listings

```
class VesselGate(nn.Module):
    def __init__(self, feat_ch, hidden=16):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Conv2d(1, hidden, 1, bias=False),
            nn.BatchNorm2d(hidden), nn.ReLU(inplace=True),
            nn.Conv2d(hidden, feat_ch, 1, bias=True))
        nn.init.zeros_(self.mlp[-1].bias)
        self.gamma = nn.Parameter(torch.zeros(1))
self.attn = None
    def forward(self, f, v):
        v = F.adaptive_avg_pool2d(v, f.shape[-2:])
        a = torch.sigmoid(self.mlp(v.to(f.dtype)))
        if not self.training:
            self.attn = a.detach()
        return f * (1.0 + self.gamma * a)
```

Code Snippet D.1: VesselGate module and VGDenseNet forward pass

```
@torch.no_grad()
def segment_vessels(gray_u8, out_size=CFG.VESSEL_CACHE_SIZE):

    P, S = CFG.VESSEL_PATCH, CFG.VESSEL_STRIDE
    x =
torch.from_numpy(gray_u8).to(DEVICE).float().div_(255).unsqueeze(1)
    B, _, H, W = x.shape
    ph, pw = (-(H - P)) % S, (-(W - P)) % S
    if ph or pw:
        x = F.pad(x, (0, pw, 0, ph), mode="reflect")
    Hp, Wp = x.shape[-2:]

    patches = F.unfold(x, kernel_size=P, stride=S)          # (B, P*P, L)
    L = patches.shape[-1]
    patches = patches.permute(0, 2, 1).reshape(B * L, 1, P, P)

    outs = []
    for k in range(0, patches.shape[0], CFG.VESSEL_PATCH_BATCH):
        logits = VESSEL_MODEL(patches[k:k + CFG.VESSEL_PATCH_BATCH])
        outs.append(torch.sigmoid(logits.float()))
    probs = torch.cat(outs).reshape(B, L, P * P).permute(0, 2, 1)

    num = F.fold(probs, (Hp, Wp), kernel_size=P, stride=S)
    den = F.fold(torch.ones_like(probs), (Hp, Wp), kernel_size=P,
stride=S)
    prob = (num / den.clamp_min(1e-6))[:, :, :H, :W]
    prob = F.interpolate(prob, size=(out_size, out_size),
                         mode="bilinear", align_corners=False)
    return (prob.clamp(0, 1) *
255).round().to(torch.uint8).squeeze(1).cpu().numpy()
```

Code Snippet D.2: Tiled sliding-window vessel segmentation

```
@torch.no_grad()
def segment_vessels(gray_u8, out_size=CFG.VESSEL_CACHE_SIZE):

    P, S = CFG.VESSEL_PATCH, CFG.VESSEL_STRIDE
    x =
torch.from_numpy(gray_u8).to(DEVICE).float().div_(255).unsqueeze(1)
    B, _, H, W = x.shape
    ph, pw = (-(H - P)) % S, (-(W - P)) % S
    if ph or pw:
        x = F.pad(x, (0, pw, 0, ph), mode="reflect")
```

```
    Hp, Wp = x.shape[-2:]

    patches = F.unfold(x, kernel_size=P, stride=S)          # (B, P*P, L)
    L = patches.shape[-1]
    patches = patches.permute(0, 2, 1).reshape(B * L, 1, P, P)

    outs = []
    for k in range(0, patches.shape[0], CFG.VESSEL_PATCH_BATCH):
        logits = VESSEL_MODEL(patches[k:k + CFG.VESSEL_PATCH_BATCH])
        outs.append(torch.sigmoid(logits.float()))
    probs = torch.cat(outs).reshape(B, L, P * P).permute(0, 2, 1)

    num = F.fold(probs, (Hp, Wp), kernel_size=P, stride=S)
    den = F.fold(torch.ones_like(probs), (Hp, Wp), kernel_size=P,
stride=S)
    prob = (num / den.clamp_min(1e-6))[:, :, :H, :W]
    prob = F.interpolate(prob, size=(out_size, out_size),
                         mode="bilinear", align_corners=False)
    return (prob.clamp(0, 1) *
255).round().to(torch.uint8).squeeze(1).cpu().numpy()build_model(rc).to(D
EVICE).to(memory_format=torch.channels_last)
    optimizer = torch.optim.AdamW(
        model.param_groups(rc.lr_backbone, rc.lr_new, rc.weight_decay,
rc.layer_decay))
    scaler = make_scaler()

    y_tr = Y["pool"][tr_idx]
    pw_full, _ = pos_weight_for(y_tr)
    curriculum = CurriculumScheduler(y_tr, epochs)
    rng = np.random.default_rng(CFG.SEED + fold)
.
    swa_model = torch.optim.swa_utils.AveragedModel(model) if rc.use_swa
else None
    swa_start = int(CFG.SWA_START_FRAC * epochs)

    train_ds = RFMiDCached("pool", tr_idx)
    sampler = IndexListSampler(np.arange(len(train_ds)))
    train_loader = DataLoader(
        train_ds, batch_size=rc.batch_size, sampler=sampler,
drop_last=True,
        num_workers=CFG.NUM_WORKERS, pin_memory=(DEVICE.type == "cuda"),
        persistent_workers=CFG.NUM_WORKERS > 0)
    train_eval_loader = make_loader("pool", tr_idx, 64)
    val_loader  = make_loader("pool", va_idx, 64)
    test_loader = make_loader("test", None, 64)
```

```
    steps = len(train_ds) // rc.batch_size
    scheduler = cosine_warmup(optimizer, steps * epochs, steps *
rc.warmup_epochs)

    best_map, best_ep, best_state, bad, history = -1.0, -1, None, 0, []
    print(f"\n=== {rc.name} | fold {fold} | {rc.backbone}@{rc.input_size}
| vessel={rc.use_vessel} "
          f"curr={rc.use_curriculum} loss={rc.loss_type} | {epochs} ep |
train {len(tr_idx)} / val {len(va_idx)} ===")

    for epoch in range(epochs):
        t0 = time.time()
        if rc.use_oversample:
            p = curriculum._class_balance_w /
curriculum._class_balance_w.sum()
            idxs = rng.choice(len(train_ds), size=len(train_ds),
replace=True, p=p)
        else:
            idxs = rng.permutation(len(train_ds))
        sampler.set(idxs)
        eff_loss = ("bce" if (rc.loss_type == "asl" and epoch <
rc.asl_warmup_frac * epochs)
                    else rc.loss_type)
        if rc.use_curriculum:
            stage, pace = curriculum.stage(epoch), curriculum.pace(epoch)
            curriculum.set_epoch_weights(epoch)
            use_sw = True
            criterion = make_criterion(eff_loss, drw_pos_weight(epoch,
epochs, pw_full),
                                       asl_gamma_neg=rc.asl_gamma_neg,
asl_clip=rc.asl_clip)
        else:
            stage, pace, use_sw = 0, 1.0, False
            criterion = make_criterion(eff_loss, pw_full,
                                       asl_gamma_neg=rc.asl_gamma_neg,
asl_clip=rc.asl_clip)

        tr_loss = train_one_epoch(
            model, train_loader, criterion, optimizer, scheduler, scaler,
            curriculum if rc.use_curriculum else None, use_sw,
            mixup_alpha=rc.mixup_alpha if rc.use_mixup else 0.0,
            mixup_prob=rc.mixup_prob if rc.use_mixup else 0.0,
            accum_steps=rc.accum_steps, strong_aug=rc.strong_aug)
```

```
        if swa_model is not None and epoch >= swa_start:
            swa_model.update_parameters(model)
        if rc.use_curriculum:
            curriculum.finalise_epoch()
            if rc.use_vessel and epoch == int(CFG.CURR_STAGE1_FRAC *
epochs) - 1:
                conf = compute_vessel_confidence(model,
train_eval_loader)
                if conf is not None:
                    curriculum.update_vessel_confidence(conf)

        _, vp, vt = predict(model, val_loader)
        m, _, _ = classification_report_ml(vt, vp)
        history.append({"epoch": epoch, "train_loss": tr_loss, "val_mAP":
m["mAP"],
                        "val_mAP_tail": m["mAP_tail"], "val_AUC":
m["macro_AUC"],
                        "sec": round(time.time() - t0, 1)})
        if m["mAP"] > best_map + 1e-4:
            best_map, best_ep, bad = m["mAP"], epoch, 0
            best_state = _state_to_cpu(model)
        else:
            bad += 1
        if epoch % CFG.LOG_EVERY == 0 or epoch == epochs - 1 or bad == 0
and epoch < 3:
            print(f"  ep {epoch:02d} | loss {tr_loss:.4f} | val mAP
{m['mAP']:.4f} (tail {m['mAP_tail']:.4f}) "
                  f"| AUC {m['macro_AUC']:.4f} | best
{best_map:.4f}@{best_ep} | {history[-1]['sec']:.0f}s")
        if rc.patience and not rc.use_swa and bad >= rc.patience:
            print(f"  early stop at epoch {epoch}: no val-mAP gain for
{rc.patience} epochs")
            break

    model.load_state_dict(best_state)
    used_swa = False
    if swa_model is not None and int(swa_model.n_averaged) > 0:
        bn_modules = [mm for mm in swa_model.modules() if isinstance(mm,
nn.modules.batchnorm._BatchNorm)]
        if bn_modules:
            momenta = {mm: mm.momentum for mm in bn_modules}
            for mm in bn_modules:
                mm.reset_running_stats(); mm.momentum = None
            swa_model.train()
            with torch.no_grad():
```

```
                for img, ves, _, _ in train_eval_loader:
                    x, v = gpu_augment(img, ves, train=False)
                    swa_model(x, v)
            for mm, mom in momenta.items():
                mm.momentum = mom
        _, vp_s, vt_s = predict(swa_model, val_loader)
        m_s, _, _ = classification_report_ml(vt_s, vp_s)
        print(f"  SWA val mAP {m_s['mAP']:.4f} vs best single epoch
{best_map:.4f}")
        if m_s["mAP"] > best_map:
            model.load_state_dict(swa_model.module.state_dict());
used_swa = True

    vz, _, vt = predict(model, val_loader)
    tz, _, _  = predict(model, test_loader)

    if fold == 0:
        sd = model.state_dict()
        if rc.backbone == "retfound":
            sd = {k: (v.half() if v.is_floating_point() else v) for k, v
in sd.items()}
        torch.save(sd, CFG.WORK_DIR / "ckpt" / f"{rc.name}_f{fold}.pt")

    out = dict(val_idx=va_idx, val_z=vz, test_z=tz, best_epoch=best_ep,
used_swa=used_swa,
               best_val_mAP=best_map, history=pd.DataFrame(history))
    del model, optimizer, scheduler, swa_model, train_loader,
train_eval_loader, val_loader, test_loader
    gc.collect(); torch.cuda.empty_cache()
    return out
```

Code Snippet D.3: Recipe dataclass defining a training run

```
def preprocess_pair(path):
    img = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if img is None:
        raise IOError(f"unreadable image: {path}")
    img = circular_fov_crop(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

    cls = cv2.resize(apply_clahe(img), (CFG.CACHE_SIZE, CFG.CACHE_SIZE),
                     interpolation=cv2.INTER_AREA)

    src = apply_clahe(img) if CFG.VESSEL_USE_CLAHE else img
```

```
    seg = cv2.resize(src, (CFG.VESSEL_INFER_SIZE, CFG.VESSEL_INFER_SIZE),
                     interpolation=cv2.INTER_AREA)
    seg = cv2.cvtColor(seg, cv2.COLOR_RGB2GRAY)
    return cls, seg
```

Code Snippet D.4: preprocess_pair with thread-local CLAHE

```
@torch.no_grad()
def segment_vessels(gray_u8, out_size=CFG.VESSEL_CACHE_SIZE):

    P, S = CFG.VESSEL_PATCH, CFG.VESSEL_STRIDE
    x =
torch.from_numpy(gray_u8).to(DEVICE).float().div_(255).unsqueeze(1)
    B, _, H, W = x.shape
    ph, pw = (-(H - P)) % S, (-(W - P)) % S
    if ph or pw:
        x = F.pad(x, (0, pw, 0, ph), mode="reflect")
    Hp, Wp = x.shape[-2:]

    patches = F.unfold(x, kernel_size=P, stride=S)          # (B, P*P, L)
    L = patches.shape[-1]
    patches = patches.permute(0, 2, 1).reshape(B * L, 1, P, P)

    outs = []
    for k in range(0, patches.shape[0], CFG.VESSEL_PATCH_BATCH):
        logits = VESSEL_MODEL(patches[k:k + CFG.VESSEL_PATCH_BATCH])
        outs.append(torch.sigmoid(logits.float()))
    probs = torch.cat(outs).reshape(B, L, P * P).permute(0, 2, 1)

    num = F.fold(probs, (Hp, Wp), kernel_size=P, stride=S)
    den = F.fold(torch.ones_like(probs), (Hp, Wp), kernel_size=P,
stride=S)
    prob = (num / den.clamp_min(1e-6))[:, :, :H, :W]
    prob = F.interpolate(prob, size=(out_size, out_size),
                         mode="bilinear", align_corners=False)
    return (prob.clamp(0, 1) *
255).round().to(torch.uint8).squeeze(1).cpu().numpy())
```

Code Snippet D.5: Tiled sliding-window vessel segmentation

```
def _affine_theta(B, train, device):
```

```
    if not train:
        th = torch.zeros(B, 2, 3, device=device)
        th[:, 0, 0] = 1.0; th[:, 1, 1] = 1.0
        return th
    ang   = (torch.rand(B, device=device) * 2 - 1) * (15 * math.pi / 180)
    scale = 1.0 + (torch.rand(B, device=device) * 2 - 1) * 0.15
    tx    = (torch.rand(B, device=device) * 2 - 1) * 0.05
    ty    = (torch.rand(B, device=device) * 2 - 1) * 0.05
    flip  = torch.where(torch.rand(B, device=device) < 0.5, -1.0, 1.0)
    cos, sin = torch.cos(ang) * scale, torch.sin(ang) * scale
    th = torch.zeros(B, 2, 3, device=device)
    th[:, 0, 0] =  cos * flip; th[:, 0, 1] = -sin; th[:, 0, 2] = tx
    th[:, 1, 0] =  sin * flip; th[:, 1, 1] =  cos; th[:, 1, 2] = ty
    return th
```

Code Snippet D.6: Joint geometric augmentation of image and vessel map

```
def param_groups(self, lr_backbone, lr_new, wd, layer_decay=1.0):

        stages = [(0, [self.stem]), (1, [self.block1, self.trans1]), (2,
[self.block2, self.trans2]),
                  (3, [self.block3, self.trans3]), (4, [self.block4,
self.norm5])]
        groups, seen = [], set()
        for lid, mods in stages:
            lr = lr_backbone * (layer_decay ** (4 - lid))
            dec  = [p for m in mods for p in m.parameters() if
p.requires_grad and p.ndim > 1]
            nodc = [p for m in mods for p in m.parameters() if
p.requires_grad and p.ndim <= 1]
            for ps, w in ((dec, wd), (nodc, 0.0)):
                if ps:
                    groups.append({"params": ps, "lr": lr,
"weight_decay": w}); seen |= {id(p) for p in ps}
        new = [p for m in (self.gates, self.classifier) for p in
m.parameters() if p.requires_grad]
        seen |= {id(p) for p in new}
        groups.append({"params": new, "lr": lr_new, "weight_decay": wd})
        assert seen == {id(p) for p in self.parameters() if
p.requires_grad}, "param_groups missed a parameter"
        return groups
```

Code Snippet D.7: VGDenseNet.param_groups (layer-wise learning-rate decay)

```
def train_one_epoch(model, loader, criterion, optimizer, scheduler,
scaler,
                    curriculum=None, use_sample_w=False,
                    mixup_alpha=0.0, mixup_prob=0.0, accum_steps=1,
strong_aug=None):
    model.train()
    tot, nb = 0.0, 0
    pbar = tqdm(loader, leave=False, desc="train")
    optimizer.zero_grad(set_to_none=True)

    for step, (img, ves, y, idx) in enumerate(pbar):
        x, v = gpu_augment(img, ves, train=True, strong=strong_aug)
        y = y.to(DEVICE, non_blocking=True)

        sw = None
        if use_sample_w and curriculum is not None:
            sw = torch.tensor(curriculum.sample_w[idx.numpy()],
                              dtype=torch.float32, device=DEVICE)

            do_mixup = mixup_alpha > 0 and np.random.rand() < mixup_prob
        with torch.autocast("cuda", enabled=CFG.AMP and DEVICE.type ==
"cuda"):
            if do_mixup:
                lam = float(np.random.beta(mixup_alpha, mixup_alpha))
                perm = torch.randperm(x.size(0), device=x.device)
                x = lam * x + (1 - lam) * x[perm]
                v = lam * v + (1 - lam) * v[perm]
                logits = model(x, v)
                loss_a, per_sample_a = criterion(logits, y, sw)
                sw_b = sw[perm] if sw is not None else None
                loss_b, per_sample_b = criterion(logits, y[perm], sw_b)
                loss = lam * loss_a + (1 - lam) * loss_b
                per_sample = lam * per_sample_a + (1 - lam) *
per_sample_b
            else:
                logits = model(x, v)
                loss, per_sample = criterion(logits, y, sw)

        scaler.scale(loss / accum_steps).backward()
        if (step + 1) % accum_steps == 0:
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(),
CFG.GRAD_CLIP)
```

```
            scaler.step(optimizer); scaler.update()
            optimizer.zero_grad(set_to_none=True)
        scheduler.step()

        if curriculum is not None:
            curriculum.update_loss(idx.numpy(), per_sample.cpu().numpy())

        tot += loss.item(); nb += 1
        pbar.set_postfix(loss=f"{tot/nb:.4f}",
lr=f"{scheduler.get_last_lr()[0]:.2e}")

    return tot / max(1, nb)
```

Code Snippet D.8: train_one_epoch with mixed precision, mixup and accumulation

```
 def fit_chain(z, y):

     ts = TemperatureScaler().fit(z, y)
     if not torch.isfinite(ts.log_T).all():
         print("   [calibration] temperature fit diverged -> identity");
 ts = TemperatureScaler().to(DEVICE)
     zt = ts(torch.as_tensor(z, dtype=torch.float32,
 device=DEVICE)).detach().cpu().numpy()
     vs = VectorScaler(z.shape[1]).fit(zt, y)
     bad = ~(torch.isfinite(vs.log_a) & torch.isfinite(vs.b))
     if bad.any():
         print(f"   [calibration] class-wise fit diverged for
 {int(bad.sum())} class(es) -> identity for those")
         with torch.no_grad():
             vs.log_a[bad] = 0.0; vs.b[bad] = 0.0
     return ts, vs
 def crossfit_probs(o):
     ks = np.unique(o["fold_of"])
     if len(ks) < 2:
         return None
     p = np.zeros_like(o["z"], dtype=np.float64)
     for k in ks:
         m = o["fold_of"] == k
         p[m] = apply_chain(fit_chain(o["z"][~m], o["y"][~m]),
 o["z"][m])
     return p
```

Code Snippet D.9: TemperatureScaler, VectorScaler and fit_chain

```
class GradCAM:
    def __init__(self, model, layer=None):
        self.model = model
        self.layer = layer if layer is not None else model.norm5
        self.acts = self.grads = None
        self.h1 = self.layer.register_forward_hook(
            lambda m, i, o: setattr(self, "acts", o))
        self.h2 = self.layer.register_full_backward_hook(
            lambda m, gi, go: setattr(self, "grads", go[0]))

    def remove(self): self.h1.remove(); self.h2.remove()

    def __call__(self, x, v, class_idx):
        self.model.eval(); self.model.zero_grad(set_to_none=True)
        with torch.enable_grad():
            logits = self.model(x, v)            # fp32: AMP off for
stable grads
            logits[:, class_idx].sum().backward()
        w = self.grads.mean(dim=(2, 3), keepdim=True)
        cam = F.relu((w * self.acts).sum(1, keepdim=True))
        cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear",
align_corners=False)
        cam = cam - cam.amin(dim=(2, 3), keepdim=True)
        cam = cam / cam.amax(dim=(2, 3), keepdim=True).clamp_min(1e-8)
        return cam.detach()


def cam_vessel_correlation(cam, vessel):
    a = cam.flatten(1);  b = vessel.flatten(1)
    a = a - a.mean(1, keepdim=True); b = b - b.mean(1, keepdim=True)
    return ((a * b).sum(1) / (a.norm(dim=1) * b.norm(dim=1) +
1e-8)).cpu().numpy()


def show_gradcam(model, split="test", class_name=None, n=4, tag="model"):
    class_name = class_name or
LABEL_COLS[int(np.argmax(Y["train"].sum(0)))]
    c = LABEL_COLS.index(class_name)
    pos = np.where(Y[split][:, c] > 0)[0][:n]
    if len(pos) == 0:
        print(f"no positives for {class_name} in {split}"); return None

    ds = RFMiDCached(split)
```

```
    img_u8 = torch.stack([ds[i][0] for i in pos])
    ves_u8 = torch.stack([ds[i][1] for i in pos])
    x, v = gpu_augment(img_u8, ves_u8, train=False)

    cam_fn = GradCAM(model)
    cam = cam_fn(x, v, c); cam_fn.remove()
    corr = cam_vessel_correlation(cam, v)

    rgb = (x * IMAGENET_STD + IMAGENET_MEAN).clamp(0, 1).cpu().permute(0,
2, 3, 1).numpy()
    fig, axes = plt.subplots(3, len(pos), figsize=(3.1*len(pos), 9),
squeeze=False)
    for j in range(len(pos)):
        axes[0, j].imshow(rgb[j])
        axes[1, j].imshow(v[j, 0].cpu(), cmap="gray")
        axes[2, j].imshow(rgb[j]); axes[2, j].imshow(cam[j, 0].cpu(),
cmap="jet", alpha=0.45)
        axes[2, j].set_title(f"corr(CAM, vessel)={corr[j]:.3f}",
fontsize=8)
        for r in range(3): axes[r, j].axis("off")
    plt.suptitle(f"{tag} -- Grad-CAM for '{class_name}'", y=0.995)
    plt.tight_layout()
    plt.savefig(CFG.WORK_DIR / f"figs/gradcam_{tag}_{class_name}.png");
plt.show()
    print(f"mean CAM-vessel correlation: {corr.mean():.4f}")
    return float(corr.mean())
```

Code Snippet D.10: GradCAM class and cam_vessel_correlation

Appendix E: Complete Results for All Recipes

Table E.1 lists every recipe, sorted by out-of-fold (OOF) mAP. OOF values pool the 2,560 development images; test values use the 640-image test split with the five-fold-model ensemble, so a recipe with fewer folds is an ensemble of fewer models. ECE columns are pooled equal-mass ECE (raw and after calibration; NaN entries for the RETFound trial mean calibration was not cross-fitted on a single fold). The loss/augmentation recipes E0 to E5 were run on three folds and the RETFound trial on one.

Table E.1: Summary of all recipes

Recipe	Folds	OOF mAP	Head	Mid	Tail	OOF AUC	ECE raw	ECE cal	Test mAP	Test AUC	Test F1	Tail F1
A_v7_regime	5	0.6340	0.7926	0.6449	0.4631	0.9416	0.0138	0.0026	0.6217	0.9541	0.5197	0.3517
E3_mixup_milder	3	0.6093	0.7838	0.6100	0.4341	0.9484	0.1585	0.0023	0.5755	0.9433	0.5051	0.2642
B_vessel_320px	5	0.6085	0.7829	0.6183	0.4232	0.9447	0.0422	0.0017	0.5936	0.9540	0.5051	0.2701
A_baseline	5	0.6069	0.7731	0.6189	0.4271	0.9408	0.0398	0.0024	0.5770	0.9496	0.5185	0.3664
B_vessel_384px	5	0.6010	0.7818	0.6171	0.4021	0.9459	0.0554	0.0020	0.6045	0.9547	0.4853	0.2206
B_vessel	5	0.5996	0.7797	0.6056	0.4127	0.9423	0.0393	0.0021	0.5766	0.9485	0.5153	0.3366
E1_asl_retuned	3	0.5982	0.7718	0.6147	0.4060	0.9473	0.1462	0.0029	0.5685	0.9432	0.4827	0.2489
E4_oversample	3	0.5905	0.7414	0.5793	0.4522	0.9259	0.2097	0.0034	0.5628	0.9118	0.4362	0.1443
E5_gentler_lr	3	0.5802	0.7714	0.5888	0.3795	0.9420	0.1879	0.0031	0.5492	0.9420	0.4699	0.2359
E0_v6_recipe	3	0.5796	0.7717	0.5989	0.3660	0.9411	0.1755	0.0015	0.5530	0.9392	0.4776	0.2331
C_vessel_curric	5	0.5757	0.7397	0.5767	0.4108	0.9178	0.0746	0.0033	0.5581	0.9221	0.4942	0.3257
E2_asl_warmup	3	0.5142	0.7158	0.5175	0.3089	0.9255	0.1418	0.0017	0.4727	0.9281	0.4258	0.1506
B_retfound	1	0.4869	0.6905	0.4497	0.3250	0.9115	0.1067	NaN	0.4412	0.8976	0.4053	0.2385
A_retfound	1	0.4833	0.6769	0.4455	0.3321	0.9078	0.1075	NaN	0.4409	0.8957	0.4010	0.1985

Head, Mid and Tail are OOF mAP by group. Test mAP, AUC, F1 and Tail F1 are for the five-model ensemble on the test split (macro-F1 and tail-class macro-F1 at OOF-fitted thresholds).

Appendix F: Per-Class Out-of-Fold Average Precision

Table F.1 gives average precision (AP) for each class from out-of-fold predictions over all 2,560 images, for the baseline, vessel-guided, curriculum and original-regime recipes. The median pooled positives per tail class is 20 (compared with 4 in the test split alone). Vessel guidance improved 11 of 25 classes and 2 of 8 tail classes relative to the baseline; the curriculum improved 4 of 25 classes; the original regime improved 19 of 25.

Table F.1: Per-class OOF average precision

Class	Group	Pooled pos.	A_baseline	B_vessel	C_vessel_curric	A_v7_regime
Disease_Risk	head	2025	0.988	0.989	0.982	0.987
DR	head	508	0.908	0.909	0.862	0.922
MH	head	419	0.866	0.874	0.861	0.875
ODC	head	354	0.722	0.731	0.649	0.757
TSLN	head	251	0.742	0.745	0.718	0.751
DN	head	184	0.411	0.398	0.327	0.420
ARMD	head	138	0.689	0.714	0.672	0.753
MYA	head	135	0.859	0.877	0.848	0.875
BRVO	mid	96	0.777	0.765	0.677	0.808
ODP	mid	91	0.367	0.389	0.318	0.407

ODE	mid	79	0.765	0.740	0.779	0.798
LS	mid	64	0.665	0.681	0.614	0.759
RS	mid	57	0.875	0.860	0.884	0.883
CSR	mid	48	0.633	0.619	0.603	0.603
CRS	mid	43	0.590	0.528	0.482	0.618
CRVO	mid	36	0.799	0.779	0.782	0.794
RPEC	mid	28	0.099	0.090	0.051	0.134
AION	tail	22	0.548	0.571	0.512	0.627
ERM	tail	21	0.103	0.070	0.081	0.063
AH	tail	20	0.735	0.692	0.794	0.809
MS	tail	20	0.188	0.128	0.105	0.127
EDN	tail	20	0.289	0.259	0.232	0.219
RT	tail	20	0.844	0.810	0.837	0.851
MHL	tail	14	0.315	0.298	0.332	0.459
PT	tail	13	0.395	0.474	0.393	0.551

Appendix G: Per-Class Test Results

Table G.1 reports per-class test metrics for the 384 px vessel-guided model (five-model ensemble, OOF-fitted calibration and per-class F1-optimal thresholds). Tail classes have only 3 to 7 test positives, so individual values are very uncertain. Three tail classes (MS, PT and RT) have zero sensitivity at their thresholds although their AUC is 0.94 to 1.00.

Table G.1: Per-class test metrics (B_vessel_384px)

Class	Group	Test pos.	Thr.	Sens.	Spec.	Prec.	F1	AUC
Disease_Risk	head	506	0.50	0.935	0.858	0.961	0.948	0.977
DR	head	124	0.46	0.798	0.973	0.876	0.835	0.982
ARMD	head	31	0.28	0.710	0.977	0.611	0.657	0.972
MH	head	104	0.44	0.875	0.950	0.771	0.820	0.969
DN	head	46	0.26	0.326	0.963	0.405	0.361	0.888
MYA	head	32	0.56	0.750	0.995	0.889	0.814	0.995
BRVO	mid	23	0.26	0.739	0.984	0.630	0.680	0.947
TSLN	head	53	0.28	0.717	0.951	0.567	0.633	0.967
ERM	tail	5	0.04	0.600	0.975	0.158	0.250	0.922
LS	mid	15	0.46	0.467	1.000	1.000	0.636	0.937
MS	tail	7	0.24	0.000	0.995	0.000	0.000	0.941
CSR	mid	13	0.32	0.308	0.997	0.667	0.421	0.977

ODC	head	91	0.28	0.692	0.918	0.583	0.633	0.909
CRVO	mid	9	0.40	0.556	1.000	1.000	0.714	0.987
AH	tail	5	0.46	0.400	0.998	0.667	0.500	0.994
ODP	mid	24	0.16	0.458	0.976	0.423	0.440	0.948
ODE	mid	17	0.44	0.706	0.998	0.923	0.800	0.975
AION	tail	4	0.24	0.250	0.998	0.500	0.333	0.937
PT	tail	4	0.36	0.000	0.998	0.000	0.000	0.975
RT	tail	5	0.50	0.000	1.000	0.000	0.000	1.000
RS	mid	14	0.20	0.643	0.990	0.600	0.621	0.988
CRS	mid	11	0.30	0.273	0.995	0.500	0.353	0.949
EDN	tail	4	0.06	0.250	0.991	0.143	0.182	0.946
RPEC	mid	4	0.16	0.000	0.992	0.000	0.000	0.925
MHL	tail	3	0.08	0.333	1.000	1.000	0.500	0.863

Appendix H: Key Formulations

The main computations are summarised below as pseudocode.

H.1 Vessel-guided residual attention gate (applied after transition 1, 2 and 3)

```
A  = sigmoid( MLP_1x1( avgpool( vessel_map ) ) )
f_out  = f * (1 + gamma * A)      # gamma learnable, initialised to 0
```

Because gamma starts at 0, training begins from the pretrained backbone, and a negative learned gamma suppresses vessel-aligned features.

H.2 Two-level curriculum difficulty (percentile-ranked signals)

```
class_difficulty  = (rarity*0.15 + cardinality*0.15) / 0.30
sample_difficulty = (loss*0.55 + misalign*0.15) / 0.70
difficulty      = (0.30*class_difficulty + 0.70*sample_difficulty)
weight(sample)  = max(0.1, sigmoid((pace - pct_difficulty)/0.12))
```

The final line is the soft pacing weight: samples easier than the current pace receive weight near 1, harder samples decay smoothly towards the floor of 0.1, and none is excluded.

H.3 Calibration and evaluation

```
logit_cal  = class_scale( logit / T )   # temperature, then class-wise
scaling
                fitted on out-of-fold logits; ranking within a class is
unchanged
ECE      = sum over equal-mass bins of (n_b / N) * |acc_b - conf_b|
paired bootstrap: resample OOF images, recompute mAP for both recipes,
report the 95% interval of the difference
```